NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 6: Recursion and Dictionaries

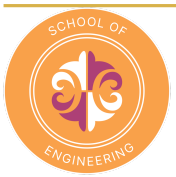
Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Reduce the problem $\longrightarrow$ Reach a base case
Rebuild the result $\longrightarrow$ Store meaning by key



About these notes

These notes accompany the 97-page presentation *Programming for Engineers: Week 6 — Recursion and Dictionaries* by Yanwei Wang. They follow the attached presentation's three lectures in order, expand its explanations, annotate its sample programs, and work through its prediction activities and exit tickets. The source presentation, rather than a generic Python syllabus, defines the scope.

Course information

Course: ENG 101 — Programming for Engineers, lecture section 2L, 6 ECTS credits.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 6: Lecture 16 — Recursion versus Iteration; Lecture 17 — Multiple Base Cases and Non-Numerical Recursion; Lecture 18 — Dictionaries and Assignment 3.

Source alignment. The grey line below each section heading identifies the corresponding *source PDF page numbers*, including title pages and lecture dividers. These are not always the same as the numbers in the slide footers. The appendix maps every source page to a reading-note section. The slide deck's own recommended readings are retained as suggested reading; they are not substituted for the attached slides.

Annotated examples. Every Python listing contains comments. Green comment text explains intent, units, control flow, assumptions, or a deliberate defect. Line numbers are reading aids: do not type them into Python. Gold boxes contain source code; grey boxes contain output. *Worked solution* and *worked extension* labels identify code developed to answer an activity or connect the slide ideas, rather than code printed verbatim in a slide.

Read errors deliberately. Listings marked *deliberate error* are not models to copy into a working program. Their separate files demonstrate `RecursionError`, `TypeError`, or `IndexError`, or a missing dictionary key. They are excluded from the combined runnable examples. Do not respond to defective recursion by increasing Python's recursion limit.

Technical clarifications. The slide examples are retained, including the course convention $F(0) = F(1) = 1$, the residual-only bisection implementation, the dictionary field names, and the labels `"safe"`/`"unsafe"`. Clearly labelled notes explain restrictions and ambiguities rather than silently changing the examples. The appendix records the main clarifications, including the Fibonacci call-tree drawing and the distinction between absent data and zero.

Assignment 3 note

Assignment 3 is introduced as a *possible course-aligned structure*, not a complete assessed brief. The slides provide no final deadline, detailed rubric, or definitive submission specification. Consult Moodle for the final task, permitted resources, collaboration rules, required disclosure, and marking criteria. Practice problems in this document are reading activities, not additional assessed requirements.

How to study Week 6

Recursion uses functions from Week 4 and often processes the strings and lists studied in Weeks 3 and 5. Dictionaries add a different organizing principle: instead of remembering which value occupies position 2, the program asks for a value by a meaningful key. These are distinct ideas, but both make a larger engineering task easier to express and inspect.

Lecture	Central question	Study emphasis
16	How does a function solve a smaller version of its own task?	Base cases, reduction, call frames, returns, factorial, iteration, and recursive bisection.
17	What happens with several stopping cases or structured inputs?	Fibonacci, repeated work, strings, palindromes, list sums, searches, and reversal.
18	How can data explain their own meaning?	Key–value records, safe lookup, named units, nested dictionaries, sensor data, and design tables.

A repeatable reading method

1. **State the contract.** Identify the input type, valid range, returned value, and relevant units before tracing any statements.
2. **Predict the base case.** Run the smallest permitted input mentally. A large recursive result depends on its terminal answers being correct.
3. **Trace both directions.** Record the calls on the way down and the returned values on the way back. A printed number and a returned number are not interchangeable.
4. **Track the objects.** For dictionaries, draw a key–value table and mark whether a statement retrieves, inserts, replaces, or removes an entry.
5. **Check the uncomfortable case.** Try an empty sequence, one element, a missing key, an input with spaces, or a reading exactly at the warning limit.
6. **Explain the evidence.** Report what was calculated, why it stopped, which rule was used, and what the calculation does not establish.

Key idea

Four recursion questions: What are the base cases? What gets smaller? Why must the calls stop? How do smaller results produce the current answer?

Four dictionary questions: What does each key mean? What type and unit does its value have? What happens when it is absent? Will this operation change authoritative data?

Using the companion files

The source package contains numbered example files, their captured outputs, and verification information. The combined `ENG101_Week6_Annotated_Examples.py` runs the working examples in sequence. Two examples ask for a material name; try `steel`, `aluminum`, or an unknown name. No external Python package is required for the teaching examples. The verification script is an optional maintenance tool, not additional Week 6 examinable material.

Scope boundary

The notes do not turn this week into a course on object-oriented programming, memoization, exception handling, advanced tree traversal, or formal asymptotic analysis. Nested structures and efficiency are discussed at the level introduced by the slides. Week 7's more systematic treatment of testing, exceptions, and assertions is previewed, not assumed.

Contents

About these notes 2

How to study Week 6 3

1. Recursion versus Iteration 7
1.1. Warm-up: repetition with a shrinking task . . . 7
1.2. What makes a function recursive? . . . 7
1.3. A first recursive countdown . . . 8
1.4. Printing before or after the recursive call . . . 10
1.5. The base case is a contract . . . 11
1.6. Two failures: no base case and the wrong direction . . . 11
1.7. Factorial: definition and implementation . . . 12
1.8. Trace the call stack and the return path . . . 13
1.9. Express the same factorial using a loop . . . 14
1.10. Validate the factorial input contract . . . 15
1.11. Worked activity: product to one . . . 16
1.12. Debug three distinct recursion defects . . . 17
1.13. Recursion depth is a resource . . . 17
1.14. Recursive bisection: the same task on a smaller interval . . . 18
1.15. Engineering interpretation: impact time . . . 20
1.16. Design checklist and Lecture 16 exit ticket . . . 20

2. Multiple Base Cases and Non-Numerical Recursion 22
2.1. Why one starting value may not be enough . . . 22
2.2. Implement the specified Fibonacci recurrence . . . 22
2.3. Trace a complete recursion tree . . . 23
2.4. Correctness and efficiency are different questions . . . 24
2.5. Worked prediction: a different two-base recurrence . . . 26
2.6. A smaller problem does not have to be a smaller number . . . 26
2.7. Palindrome testing as a recursive question . . . 27
2.8. Short-circuit evaluation avoids unnecessary recursion . . . 28
2.9. Separate normalization from structural recursion . . . 28
2.10. The first-element plus remaining-list pattern . . . 30
2.11. Recursive search with two stopping cases . . . 31
2.12. Debug the non-numerical recursion . . . 32
2.13. A general recipe and recursive reversal . . . 33
2.14. Lecture 17 exit ticket: count a letter . . . 34

3. Dictionaries and Assignment 3 35
3.1. Warm-up: what does position 2 mean? . . . 35
3.2. Create and inspect the first dictionary . . . 35
3.3. Keys, values, and a consistent vocabulary . . . 36
3.4. Add or update an entry . . . 37
3.5. Direct access and the missing-key error . . . 38
3.6. Test membership before retrieval . . . 38
3.7. Use get when a default has a clear meaning . . . 39
3.8. Keys, values, items, and iteration . . . 40
3.9. Remove entries deliberately . . . 42

3.10. An interactive material-density lookup . . . 43
3.11. Nested dictionaries for related properties . . . 43
3.12. Normalize the outer name, then inspect the record . . . 44
3.13. Sensor metadata and repeated readings . . . 45
3.14. Evaluate a list of design dictionaries . . . 46
3.15. Choose among a list, tuple, and dictionary . . . 48
3.16. Debug a failed nested lookup . . . 48
3.17. A dictionary design checklist . . . 50
3.18. Assignment 3: preparation, not a replacement brief . . . 50
3.19. Connect recursion and dictionaries . . . 51
3.20. Lecture 18 exit ticket: a pressure-sensor record . . . 52

4. Synthesis, Quick Reference, and Worked Practice 54
4.1. The Week 6 reasoning pattern . . . 54
4.2. Dictionary quick reference . . . 55
4.3. A Week 6 glossary . . . 55
4.4. Cumulative practice questions . . . 57
4.5. Worked solutions . . . 58
4.6. Readiness checklist for the next week . . . 64

A. Technical Clarifications and Source Boundaries 65
A.1. The integer check accepts Boolean flags . . . 65
A.2. Unique keys and integer-key access . . . 65
A.3. Views and snapshots are different objects . . . 66
A.4. An outer copy does not duplicate a nested list . . . 67
A.5. Clarification register . . . 67
A.6. What remains outside the supplied material . . . 68

B. Source Map and Suggested Reading 69
B.1. How the source was used . . . 69
B.2. Complete slide-to-notes map . . . 69

Bibliography 71

1 Recursion versus Iteration

Lecture-slide alignment: Lecture 16, source PDF pp. 3–31.

Learning outcomes

After this chapter, you should be able to identify base and recursive cases, trace calls and returns, state a termination argument, compare factorial implementations, test recursive inputs, diagnose common defects, and explain the recursive form of bisection.

1.1. Warm-up: repetition with a shrinking task

Lecture-slide alignment: Week 6 source PDF, pp. 3–5. PDF page numbers include title and divider pages.

The opening activity asks about a countdown, factorial, and repeated interval halving. All involve repetition, but the important observation is that *something becomes smaller*. Counting down from 5 approaches a stopping number. Evaluating 5! reduces the task to evaluating 4!. Bisecting an interval reduces the range still under consideration.

Task	Stopping idea	What becomes smaller?
Countdown	Reach zero, then print the final message.	A nonnegative integer count.
Factorial	Use the known value $0! = 1$.	The factorial argument n.
Bisection	Satisfy a specified numerical accuracy criterion.	The retained interval width.

A shrinking task is not yet a complete program. We still need a direct answer for the terminal case and an explanation of what each waiting call will do with the smaller answer. In bisection we must also state what "sufficiently accurate" means. The function later in this chapter uses residual accuracy, even though the opening discussion mentions interval width too.

1.2. What makes a function recursive?

Lecture-slide alignment: Week 6 source PDF, pp. 6–7. PDF page numbers include title and divider pages.

A **recursive function** calls itself, directly or through another function, while solving a smaller or simpler instance of the same problem. The slide examples use direct recursion: the function's own name appears in a call inside its body.

The **base case** is an input for which the result is available immediately, without another recursive call. The **recursive case** handles a remaining valid input by reducing it, asking the same function to solve the smaller task, and using the returned information to complete the current task.

Definition

Recursion is not just repeated function calling. A successful recursive design includes an input contract, reachable base cases, a reduction toward them, and a rule for using returned results.

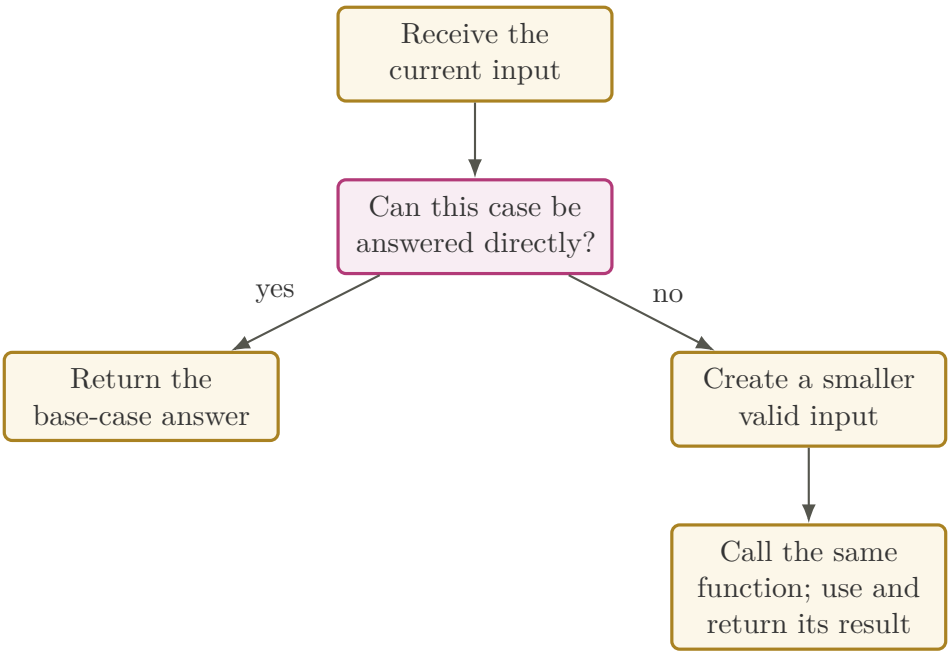


Figure 1.1.: The two-part pattern from source PDF p. 7, redrawn as a reading aid. Every route must either finish or lead to a smaller valid problem.

The call does not overwrite the current call's local variables. Each call has its own parameters and local workspace, just as in Week 4. The older call can wait while a new call works on another input. This explains both the usefulness of recursion and the extra memory it can require.

1.3. A first recursive countdown

Lecture-slide alignment: Week 6 source PDF, pp. 8–9. PDF page numbers include title and divider pages.

The first program prints a countdown and a launch message. It does not calculate and return a numerical answer.

Annotated slide example. File: `ex01_countdown.py`.

```python
# The input is the countdown value; use a small integer for tracing.
def countdown(n):
    # Base case: zero or a negative input launches immediately.
    if n <= 0:
        # Printing communicates a message; it does not return the message.
        print("Launch")
        # A bare return finishes this call and returns None.
        return
    # This output happens on the way down, before the next call.
    print(n)
    # Pass a smaller value; the current call waits for this call to finish.
    countdown(n - 1)

# Define first, then call: the visible output is 3, 2, 1, Launch.
countdown(3)
```

Code 1.1: A countdown with a reachable base case.

```
3
2
1
Launch
```

The definition creates the function; `countdown(3)` starts it. In the first call, `n` is 3, so the base-case test is false. The program prints 3, then starts a second call whose own `n` is 2. The same pattern produces calls with 1 and 0. At zero, the function prints `"Launch"` and executes the bare `return`.

Call	Local n	Immediate action	Next action
1	3	Print 3.	Call `countdown(2)`.
2	2	Print 2.	Call `countdown(1)`.
3	1	Print 1.	Call `countdown(0)`.
4	0	Print `Launch`.	Return directly; no fifth call.

The base-case return ends *that call*, not the entire Python program. Control goes back to the waiting call with `n=1`; it has no further statements and also finishes. This repeats until the original call finishes. Each returns `None`. The visible countdown comes from `print()`, not from returned values.

Stop, predict, and try it

Predict `countdown(0)` and `countdown(-2)`. In this exact example, both print `Launch` immediately because the test is `n <= 0`. This behavior is part of the chosen countdown rule; it does not imply that negative inputs are valid in every recursive algorithm.

1.4. Printing before or after the recursive call

Lecture-slide alignment: Week 6 source PDF, p. 10. PDF page numbers include title and divider pages.

The position of a statement relative to the recursive call determines *when* it runs. Code before the call executes during descent. Code after the call must wait until the smaller call returns, so it executes during unwinding.

Annotated slide example. File: `ex02_output_order.py`.

```python
# Both versions assume a small, nonnegative integer input.
def show_a(n):
    # At zero, finish without printing another number.
    if n == 0:
        return
    # Print during descent: larger values appear first.
    print(n)
    # Reduce the input by one.
    show_a(n - 1)

# Use the same stopping case but reverse the two remaining actions.
def show_b(n):
    # The base case contributes no printed number.
    if n == 0:
        return
    # Finish the smaller problem before printing the current value.
    show_b(n - 1)
    # Print during unwinding: smaller values appear first.
    print(n)

# Added labels separate the two prediction answers.
print("Version A:")
show_a(3)
print("Version B:")
show_b(3)
```

Code 1.2: The same smaller input, but two different output orders.

```
Version A:
3
2
1
Version B:
1
2
3
```

For `show_b(3)`, the call with 3 starts the call with 2 before printing anything. That call starts the call with 1, which starts the terminal call with 0. Only then does printing begin: the waiting call with 1 prints first, followed by 2 and 3. The values of `n` have not all become zero. Each waiting call retains its own value.

Moving `print(n)` below the recursive call in the original countdown similarly gives `Launch`, then 1, 2, and 3. The base-case message appears before the waiting calls resume.

Common mistake

Do not trace a recursive function by simply reading its source from top to bottom once. Every function call starts at the beginning of the function body with a new local input. Every return resumes an earlier unfinished call.

1.5. The base case is a contract

Lecture-slide alignment: Week 6 source PDF, p. 11. PDF page numbers include title and divider pages.

A base case must satisfy three questions. Is the stopping input reachable from every valid starting input? Is the direct returned result correct? Do the base cases cover every possible terminal form? A condition that merely appears somewhere in the function is not sufficient.

For factorial on nonnegative integers, a useful progress measure is n. Each nonterminal call replaces it with $n - 1$, which remains a nonnegative integer until zero is reached. Starting from $n = 4$, only four reductions are possible before the base case. The sum of a list uses the number of remaining elements as an analogous measure.

Technical clarification: a decreasing measure needs a stopping guarantee

Source PDF p. 11 describes a nonnegative quantity that strictly decreases. For the integer counts and sequence lengths used here, this gives a finite argument when every nonterminal step decreases by at least one. A positive real number can decrease forever without reaching zero: $1, 1/2, 1/4, \ldots$. Numerical recursion therefore needs a positive tolerance or another finite stopping rule, not merely a shrinking real interval.

The valid input domain also matters. The versions `show_a` and `show_b` use `n == 0`; starting at a negative integer and repeatedly subtracting one would never reach that base case. Their introductory contract is therefore a small nonnegative integer. A contract states what is permitted, not just what happens in the one demonstration call.

1.6. Two failures: no base case and the wrong direction

Lecture-slide alignment: Week 6 source PDF, pp. 12–13. PDF page numbers include title and divider pages.

The first faulty function keeps calling itself even after passing zero.

Annotated slide example. File: `ex03_no_base.py`.

```
# DELIBERATE ERROR: study separately; this function never stops itself.
def broken_countdown(n):
    # Output continues past zero because no test ends the recursion.
    print(n)
    # Decreasing a value does not help without a reachable stopping rule.
    broken_countdown(n - 1)

# Python eventually raises RecursionError; this is not successful completion.
broken_countdown(3)
```

Code 1.3: Deliberate error: a smaller number without a stopping condition.

A typical final exception is `RecursionError: maximum recursion depth exceeded`. The

exact traceback and the number of printed lines depend on the Python environment. The essential fact is that the algorithm did not decide to finish; Python interrupted an excessively deep chain of active calls.

The second faulty function contains a base case, but its update moves positive input away from that case.

Annotated slide example. File: `ex04_wrong_direction.py`.

```
# DELIBERATE ERROR: the base case cannot be reached from positive n.
def countdown(n):
    # This test exists, but the recursive update moves away from it.
    if n == 0:
        return
    # Print the increasing value to reveal the direction of progress.
    print(n)
    # Wrong update: n + 1 is larger, not closer to zero.
    countdown(n + 1)

# Added demonstration call: expect RecursionError, not termination at zero.
countdown(3)
```

Code 1.4: Deliberate error: a base case that is approached in the wrong direction.

For a starting value of 3, the inputs are 3, 4, 5, and so on. Changing the recursive argument from `n + 1` to `n - 1` repairs the direction for a nonnegative-integer contract. Input validation or a different terminal rule is still needed if other inputs are intended.

Design principle

Diagnose the defect before editing. Write the actual sequence of recursive arguments. Then identify whether the missing ingredient is a stopping condition, a reachable stopping condition, or a valid input assumption.

1.7. Factorial: definition and implementation

Lecture-slide alignment: Week 6 source PDF, pp. 14–15. PDF page numbers include title and divider pages.

For a nonnegative integer n, the slides define

$$0! = 1, \qquad n! = n(n-1)! \quad \text{for } n > 0.$$

The base case is one because one is the neutral value for multiplication. If the base case returned zero, every larger product would become zero. The recursive formula tells us both how to reduce the input and how to combine the smaller result.

Annotated slide example. File: `ex05_factorial_recursive.py`.

```python
# This first version assumes that n is a small nonnegative integer.
def factorial_recursive(n):
    """Return n! for a nonnegative integer n."""
    # Base case: the empty product is one, not zero.
    if n == 0:
        return 1
    # Wait for (n - 1)!, multiply by this call's n, and return the product.
    return n * factorial_recursive(n - 1)

# A hand-checkable call: 5 * 4 * 3 * 2 * 1 = 120.
print(factorial_recursive(5))
```

Code 1.5: The mathematical factorial definition expressed as Python.

```
120
```

In `return n * factorial_recursive(n - 1)`, Python cannot finish the multiplication until the smaller call returns. The local value of `n` is preserved during that wait. The `return` keyword then sends the complete product to the next waiting caller. A docstring records the intended domain but does not enforce it automatically.

1.7.1. Why the direct result is a returned value

The program should be able to store the factorial in a variable or use it in a later expression. Returning the value makes that possible. Printing the product inside the function would only display it; the caller would not receive that numerical result unless the function also returned it. This is exactly the distinction developed in Week 4.

1.8. Trace the call stack and the return path

Lecture-slide alignment: Week 6 source PDF, pp. 16–18. PDF page numbers include title and divider pages.

To evaluate `factorial_recursive(4)`, first expand the pending work:

$$\begin{aligned} 4! &= 4\,(3!) \\ &= 4 \cdot 3\,(2!) \\ &= 4 \cdot 3 \cdot 2\,(1!) \\ &= 4 \cdot 3 \cdot 2 \cdot 1\,(0!). \end{aligned}$$

There are five active factorial calls at the deepest point, including the call with zero. The **call stack** holds the unfinished work. A **frame** is the local context of one active call: its arguments, local names, and where to resume after another call finishes.

Local input	What the call waits for	What it returns
0	Nothing; the base case is direct.	1
1	The result of the call with 0.	$1 \times 1 = 1$
2	The result of the call with 1.	$2 \times 1 = 2$
3	The result of the call with 2.	$3 \times 2 = 6$
4	The result of the call with 3.	$4 \times 6 = 24$

Table 1.1.: Return-path trace corresponding to the stack and unwinding diagrams on source PDF pp. 17–18. Read from the base case upward.

The most recently started unfinished call normally completes first in this simple chain. This is often described as last in, first out. The call with 4 does not need to recompute everything after the base case; it resumes its already pending multiplication when the call with 3 supplies 6.

Trace the program

Draw a separate row for every *call*, not for every function name. All five frames execute the same function definition, but they have different inputs and different unfinished multiplications. The source code is shared; the local execution contexts are not.

1.9. Express the same factorial using a loop

Lecture-slide alignment: Week 6 source PDF, pp. 19–21. PDF page numbers include title and divider pages.

A loop builds a running product in one function call instead of leaving a multiplication pending in every frame.

Annotated slide example. File: `ex06_factorial_iterative.py`.

```python
# The same valid-input contract is used for the loop version.
def factorial_iterative(n):
    """Return n! for a nonnegative integer n."""
    # One is the neutral starting value for a product.
    result = 1
    # Include n by choosing the excluded stopping value n + 1.
    for value in range(1, n + 1):
        # Multiply the running product by the next positive integer.
        result *= value
    # Return after the entire loop, not during its first pass.
    return result

# The result agrees with the recursive version for this valid input.
print(factorial_iterative(5))
```

Code 1.6: Iterative factorial with an accumulator.

```
120
```

For $n = 4$, the accumulator changes from 1 to 1, then 2, then 6, and finally 24. For $n = 0$, `range(1, 1)` produces no loop values, so the function returns the correctly initialized product 1. Zero loop passes can therefore be correct behavior, not a failure.

Question	Iteration	Recursion
What repeats?	A loop body.	A function call.
Where is progress stored?	Updated variables in one call.	Local information in active frames.
What ends the work?	The sequence ends or a loop condition becomes false.	A base case returns directly.
How is progress argued?	State approaches a specified limit.	Every recursive input becomes simpler.
Typical defect	Infinite loop or wrong accumulator.	Missing base case, wrong reduction, or missing return.
Natural use	Sequential calculations and long data scans.	Self-similar definitions and suitable nested tasks.

The slides do not propose recursion as automatically superior. For a long measurement series, a loop avoids one active recursive call per element. A recursive formulation can instead make a self-similar problem easier to explain. Choose the representation that makes correctness, assumptions, stopping, and resource requirements easiest to inspect.

1.10. Validate the factorial input contract

Lecture-slide alignment: Week 6 source PDF, pp. 22–23. PDF page numbers include title and divider pages.

A negative input repeatedly reduced by one does not reach zero. A value such as 2.5 passes through 1.5, 0.5, and negative half-integers, also missing zero. The slide therefore adds an explicit type and range test before the base case.

Annotated slide example. File: `ex07_factorial_validated.py`.

```python
# The validator below retains the exact test used in the slide.
def factorial_recursive(n):
    """Return n! for valid n; return None for a rejected input."""
    # Short-circuit or avoids n < 0 when n is not an int.
    if not isinstance(n, int) or n < 0:
        # None is a failure indicator, not a numerical factorial.
        return None
    # Base case for a valid nonnegative integer.
    if n == 0:
        return 1
    # Pass a smaller integer and return the complete product.
    return n * factorial_recursive(n - 1)

# Added calls implement the base, near-base, known, and invalid tests.
for n in [0, 1, 5, -1, 2.5]:
    # Report the input and the observed returned value.
    print(n, "->", factorial_recursive(n))
```

Code 1.7: The slide's validation policy, followed by its proposed test cases.

```
0 -> 1
1 -> 1
5 -> 120
-1 -> None
2.5 -> None
```

`isinstance(n, int)` asks whether `n` belongs to Python's integer type or a subclass of it. The logical `or` short-circuits: if the first condition already says the input is not an integer, Python need not evaluate `n < 0`. This matters for an input such as `"five"`, for which comparison with zero would be inappropriate.

`None` is an explicit failure indicator here. A caller receiving it should report invalid input rather than multiply it, add to it, or present it as a numerical factorial. A checked contract is still not a promise to handle arbitrarily large valid integers recursively; call depth remains a separate resource limitation.

Technical clarification: Boolean flags pass this exact slide check

Python's `bool` type is a subclass of `int`. Consequently the slide's `isinstance(n, int)` test accepts `True` and `False`; both produce 1 in this function. The teaching code is retained unchanged. A stricter application can explicitly exclude Boolean values, as demonstrated in Appendix A. This is a refinement of the input policy, not a different factorial formula. See the supplementary Python documentation reference [5].

1.10.1. Test the base case before the large case

The tests in the slide check zero, one, a known result, a negative boundary, a non-integer, and agreement with an iterative version. Each supplies different evidence. Agreement between two implementations is useful, but independently known values are stronger than agreement alone: two functions can share a mistake. A small table of 0! through 5! appears in the review exercises.

1.11. Worked activity: product to one

Lecture-slide alignment: Week 6 source PDF, p. 24. PDF page numbers include title and divider pages.

The activity uses a slightly different stopping rule, `n <= 1`. Trace that actual rule rather than importing the earlier zero-only trace.

Annotated slide example. File: `ex08_product_trace.py`.

```python
# This activity deliberately uses n <= 1 as its terminal condition.
def product_to_one(n):
    # For the intended nonnegative integers, 0 and 1 both return 1.
    if n <= 1:
        return 1
    # Multiply only after the smaller call has returned its answer.
    return n * product_to_one(n - 1)

# The slide asks for the value stored in answer.
answer = product_to_one(4)
# Added display makes the activity result visible when this file is run.
print(answer)
```

Code 1.8: The source activity and an added display of its answer.

The calls are `product_to_one(4)`, then 3, then 2, then 1. The last call returns 1 directly; there is no call with zero. The returns are 1, 2, 6, and 24. The variable `answer` stores 24.

For the intended nonnegative integer factorial task, the two base-case choices agree. However, the `n <= 1` version also returns 1 for negative inputs, which is not validation of a nonnegative-integer

factorial contract. Changing a stopping rule can change behavior outside the intended domain. Record the domain and the rule separately.

1.12. Debug three distinct recursion defects

Lecture-slide alignment: Week 6 source PDF, p. 25. PDF page numbers include title and divider pages.

The slide's first defect, `return n * f(n-1)` with no base case, leaves every multiplication waiting for another call. The second uses `n+1` after a zero base case, so positive inputs move in the wrong direction. Both eventually exceed the permitted call depth. They are multiplication versions of the countdown defects already examined.

The third defect is subtler: the recursive case computes an expression but does not return it.

Annotated slide example. File: `ex09_missing_return.py`.

```
# DELIBERATE ERROR: the recursive-case expression is not returned.
def f(n):
    # Only the base-case path explicitly returns a value.
    if n == 0:
        return 1
    # For n = 1 this expression is computed and discarded.
    n * f(n - 1)
    # Falling off the function returns None.

# The first call prints None; the second attempts 2 * None.
print(f(1))
print(f(2))  # Raises TypeError rather than producing a factorial.
```

Code 1.9: Deliberate error: the smaller answer is not returned through the full chain.

For `f(1)`, the smaller call `f(0)` returns 1. The expression 1×1 is evaluated and discarded, then the function reaches its end and returns `None`. For `f(2)`, the inner call therefore supplies `None`; evaluating `2 * None` raises a `TypeError`. For larger inputs the same problem appears while unwinding. It is not correct to say that every nonzero input simply returns `None` without an exception.

Answer and explanation

The smallest intended repair is to add `return` before the recursive-case multiplication. Then check all three elements together: a correct base result, a smaller valid argument, and a returned combination. A successful base-case test alone would not reveal this missing-return defect.

1.13. Recursion depth is a resource

Lecture-slide alignment: Week 6 source PDF, p. 26. PDF page numbers include title and divider pages.

Every active call retains enough information to resume. A mathematically terminating function can therefore exceed the permitted depth before it reaches its base case. Mathematical termination and practical execution are different questions.

The factorial chain for n uses calls for $n, n-1, \ldots, 0$. A recursive sum over a list similarly needs a call for every shorter remainder. On a very long dataset, such a linear chain is usually a poor

implementation choice in Python even when its logic is correct. A loop can retain a small running state without leaving one call pending for each sample.

Common mistake

`RecursionError` is diagnostic information, not an invitation to raise the limit until the program stops complaining. First check input validity, base cases, reduction, and whether a loop is the better design. The exact permitted depth is environment-dependent; these notes do not promise a fixed universal value. Python documents the limit as a safeguard [8].

1.14. Recursive bisection: the same task on a smaller interval

Lecture-slide alignment: Week 6 source PDF, pp. 27–28. PDF page numbers include title and divider pages.

Week 3 used bisection to solve a target equation $f(x) = 0$. On an interval $[a, b]$, the method calculates $m = (a + b)/2$ and retains a half-interval that still brackets a root. The next task has exactly the same structure as the original one, but the interval is half as wide. That self-similarity motivates the recursive formulation.

Function contract

For the teaching function below, assume a callable residual function, a suitable interval with opposite endpoint signs, continuity on the interval, and a positive residual tolerance. The implementation does *not* enforce these assumptions, check an interval-width tolerance, recognize all endpoint-root cases explicitly, or impose a maximum number of calls.

Annotated slide example. File: `ex10_bisection.py`.

```python
# This is the residual-only teaching algorithm from the slides.
def bisection_recursive(function, left, right, tolerance):
    # Find the middle of the current interval.
    midpoint = (left + right) / 2
    # Base case: accept a small residual, in the units of function's output.
    if abs(function(midpoint)) < tolerance:
        return midpoint
    # If left and midpoint have opposite signs, retain the left half.
    if function(left) * function(midpoint) < 0:
        # Return the smaller call's answer to the caller above it.
        return bisection_recursive(
            function, left, midpoint, tolerance)
    # Otherwise retain the right half of the assumed valid bracket.
    return bisection_recursive(
        function, midpoint, right, tolerance)

# The input is time in seconds; the residual is height in metres.
def residual_time(t_s):
    # Model assumption: constant gravity, initial rest, no air resistance.
    g = 9.81  # Gravitational acceleration in m/s^2.
    # Height above ground for release from 20 m.
    return 20.0 - 0.5 * g * t_s**2

# The endpoints 0 s and 3 s give opposite residual signs.
impact_time_s = bisection_recursive(
    residual_time, left=0.0, right=3.0,
    tolerance=1e-6)  # This tolerance is in metres, not seconds.
# Four decimal places are an output format, not an error guarantee.
print(f"Impact time = {impact_time_s:.4f} s")
```

Code 1.10: The source's residual-only recursive bisection and the falling-object application.

The parameter `function` receives a function object. Passing `residual_time` without parentheses gives the bisection algorithm a tool it can call at different trial times. Writing `residual_time(...)` instead would call it immediately and supply its numerical result, which is not the same interface.

The first `if` is the base case: a small absolute residual returns the midpoint. If it is not satisfied, the endpoint-sign test chooses one of two recursive paths. Only one of those paths is executed during a call. The method is not making both half-interval calls and adding the results; it is selecting the half that still contains the relevant root evidence.

Technical clarification: the slide algorithm has only residual-based stopping

Source PDF p. 27 mentions residual *or* interval stopping in its verbal outline. The actual function on p. 28 checks only `abs(function(midpoint)) < tolerance`. These notes preserve that code and distinguish it from the broader outline. A shrinking interval alone does not guarantee success in finite-precision arithmetic for an unrealistically small residual tolerance. A more defensive solver would validate inputs, handle endpoint roots, monitor interval width and midpoint stagnation, and limit the number of reductions. Those checks are not present in this teaching implementation.

1.15. Engineering interpretation: impact time

Lecture-slide alignment: Week 6 source PDF, p. 29. PDF page numbers include title and divider pages.

The example releases an object from rest at height 20 m under constant gravitational acceleration $g = 9.81\,\mathrm{m\,s^{-2}}$, neglecting air resistance. Its model is

$$h(t) = 20 - \tfrac{1}{2}gt^2.$$

Impact corresponds to the first positive time at which the modeled height is zero. With t in seconds, `residual_time(t)` returns height in metres. At the starting endpoints,

$$h(0) = 20 \text{ m}, \qquad h(3) = 20 - \tfrac{1}{2}(9.81)(9) = -24.145 \text{ m}.$$

The polynomial model is continuous, and these signs bracket its positive root. Negative modeled heights during numerical trials indicate that a trial time lies beyond the model's ground-crossing event; they do not represent a simulation of the object's later interaction with the ground.

```
Impact time = 2.0193 s
```

1.15.1. A worked independent check

Solving the same simplified equation analytically gives

$$t_* = \sqrt{\frac{2(20)}{9.81}} \approx 2.019275109 \text{ s}.$$

This is an independent calculation within the same model, not experimental validation of the model. The function accepts a height residual below 10^{-6} m. Its input is a time, but the tolerance is *not* 10^{-6} s. The four digits after the decimal point in the printed result are a display choice; rounding is separate from the convergence test.

Engineering check

State the model, the bracket, the tolerance quantity and unit, and the numerical evidence before interpreting the time. Recursion changes how the algorithm is organized; it does not remove air resistance, measurement uncertainty, or the need to judge whether the falling-object model is suitable.

1.16. Design checklist and Lecture 16 exit ticket

Lecture-slide alignment: Week 6 source PDF, pp. 30–31. PDF page numbers include title and divider pages.

A complete explanation of a recursive function starts with the valid input domain and expected output. Write every base case, identify a finite progress measure, check every recursive path, and state how the returned sub-results become the current result. Trace a tiny example including its return path. Test base, near-base, normal, and invalid inputs, then compare with iteration where that clarifies the design.

The exit ticket defines

$$R(n) = \begin{cases} 0, & n = 0, \\ n + R(n-1), & n > 0. \end{cases}$$

Its base case is $R(0) = 0$; its integer argument decreases by one. Expanding $R(3)$ gives $3 + 2 + 1 + 0 = 6$. The function calculates $1 + 2 + \cdots + n$ for a nonnegative integer n, rather than a factorial.

Worked slide activity solution. File: `ex11_exit16.py`.

```
# Worked solution: R(0) = 0 and R(n) = n + R(n - 1).
def sum_to_n(n):
    # Assumption: n is a small nonnegative integer.
    if n == 0:
        # Zero is the neutral value for addition.
        return 0
    # Combine the current integer with the smaller partial sum.
    return n + sum_to_n(n - 1)

# R(3) = 3 + 2 + 1 + 0 = 6.
print(sum_to_n(3))
```

Code 1.11: Worked solution to the Lecture 16 exit ticket.

A loop would normally be preferable to one recursive call per integer in a large Python calculation. A direct mathematical formula $n(n + 1)/2$ is another independent check on the recurrence, when that sum is the actual task. The choice should follow the problem, not a requirement to force recursion into every repeated calculation.

Lecture 16 in one paragraph

A recursive function solves a task by reaching a direct base answer through smaller valid inputs, then returning the information needed by its callers. The call stack preserves unfinished work. Factorial reveals the descent and the unwinding particularly clearly. Input validity, reachable stopping, correct returns, and resource limits must all be checked. Bisection shows that a familiar engineering algorithm can have a recursive structure, while still depending on its numerical and physical assumptions.

2 Multiple Base Cases and Non-Numerical Recursion

Lecture-slide alignment: Lecture 17, source PDF pp. 32–62.

Learning outcomes

After this chapter, you should be able to explain multiple base cases, trace several recursive calls in one expression, compare recursive and iterative Fibonacci, process strings and lists recursively, use short-circuit logic, and repair a missing terminal case or a failure to shrink the input.

2.1. Why one starting value may not be enough

Lecture-slide alignment: Week 6 source PDF, pp. 32–35. PDF page numbers include title and divider pages.

The opening activity presents the sequence

$$1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13, \ldots$$

and asks why two starting values are needed. Every new term is the sum of the previous *two* terms. A rule involving two previous values cannot calculate the next term unless both are available. The slides therefore use two base inputs:

$$F(0) = 1, \qquad F(1) = 1, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

Key idea

Keep the course convention. In this presentation the sequence is indexed from zero and starts with $F(0) = F(1) = 1$. Consequently $F(4) = 5$ and $F(5) = 8$. Do not change a starting value while tracing the given code; the initial values are part of the specification.

Multiple base cases do not necessarily mean multiple `if` statements. The two input conditions can share one branch when they return the same value, as in `n == 0 or n == 1`. What matters is that every smallest valid input reached by the recursive rule has an appropriate direct answer.

There are two differences from factorial. Fibonacci has two base inputs, and its nonterminal expression evaluates two smaller recursive calls. It therefore develops a branching pattern of calls rather than one simple chain. Each branch must still move toward a base case.

2.2. Implement the specified Fibonacci recurrence

Lecture-slide alignment: Week 6 source PDF, p. 36. PDF page numbers include title and divider pages.

Annotated slide example. File: `ex12_fibonacci.py`.

```python
# The course convention is F(0) = 1 and F(1) = 1.
def fibonacci(n):
    """Return F(n) for an integer n >= 0; use small n here."""
    # Two input cases have a direct answer of one.
    if n == 0 or n == 1:
        return 1
    # Both smaller calls must return before their results can be added.
    return fibonacci(n - 1) + fibonacci(n - 2)

# The sequence indexed from zero is 1, 1, 2, 3, 5, 8, ...
print(fibonacci(5))
```

Code 2.1: Recursive Fibonacci with the exact starting values from the slides.

```
8
```

The contract is a small nonnegative integer input. The docstring states that domain, but this introductory function does not check it. With $n \geq 2$, both $n - 1$ and $n - 2$ are nonnegative and smaller than n. Starting with a valid input therefore leads through a finite collection of smaller integer problems to 0 or 1.

For `fibonacci(2)`, the program calls `fibonacci(1)` and `fibonacci(0)`. Each returns 1, so the call returns 2. For `fibonacci(3)`, it adds the result 2 from `fibonacci(2)` to the result 1 from `fibonacci(1)`, giving 3.

The addition requires two returned values. Python evaluates the left recursive expression, then the right one, and only then combines them. The fact that a call to `fibonacci(1)` occurred earlier does not mean Python automatically reuses that result on a later call.

2.3. Trace a complete recursion tree

Lecture-slide alignment: Week 6 source PDF, pp. 37–38. PDF page numbers include title and divider pages.

A **recursion tree** records individual call occurrences. The children of a node are the smaller calls made while evaluating that occurrence. Different nodes may carry the same argument because the same subproblem is recomputed.

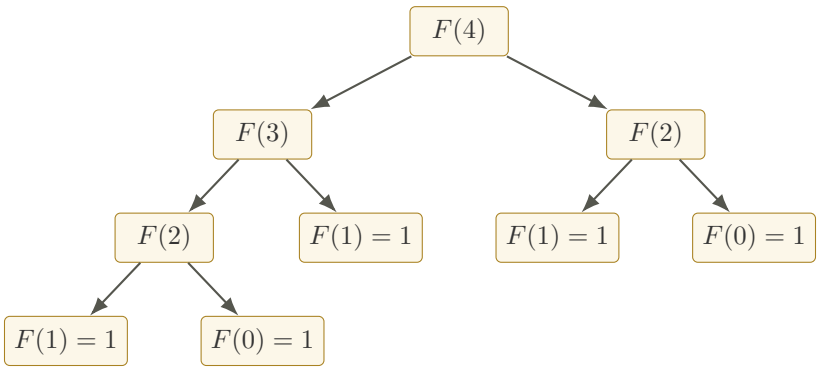


Figure 2.1.: Complete call-occurrence tree for the supplied `fibonacci(4)` implementation: nine calls, including five base-case calls. Identical arguments do not imply a shared execution.

Diagram clarification: calls are not automatically shared

The diagram on source PDF p. 37 connects two branches to a shared-looking $F(1)$ node. The supplied Python function does not cache or share computed results. The expanded tree above draws those occurrences separately so that each call has one parent. This makes the slide's warning about repeated work explicit; it does not change the recurrence or introduce memoization.

Result	Combination	Returned value
$F(0)$	Direct base result.	1
$F(1)$	Direct base result.	1
$F(2)$	$F(1) + F(0)$	$1 + 1 = 2$
$F(3)$	$F(2) + F(1)$	$2 + 1 = 3$
$F(4)$	$F(3) + F(2)$	$3 + 2 = 5$
$F(5)$	$F(4) + F(3)$	$5 + 3 = 8$

This table summarizes numerical values; it is not a record of every function invocation. The tree answers "How many times did we do a task?" while the table answers "What value does that task produce?" Both are useful, but they should not be confused.

2.4. Correctness and efficiency are different questions

Lecture-slide alignment: Week 6 source PDF, pp. 39–41. PDF page numbers include title and divider pages.

The recursive code is attractive because it closely mirrors the mathematical recurrence. Its base cases and combination rule are visible. However, as n increases, its call tree contains many repeated subproblems. Correct arithmetic and a valid termination argument do not make that repeated work economical.

The slides introduce this observation without requiring formal complexity notation. At this stage, identify repeated work, notice memory used by active frames, and compare a simpler alternative. Formal efficiency analysis comes later in the course.

2.4.1. An iterative alternative

Only two previous sequence values are needed to calculate the next one. The iterative algorithm stores those two values and updates them once per new term.

Annotated slide example. File: `ex13_fibonacci_iterative.py`.

```python
# Compute the same F(n), with F(0) = F(1) = 1.
def fibonacci_iterative(n):
    # Assumption: n is a nonnegative integer.
    if n == 0 or n == 1:
        return 1
    # Initialize the two values needed to calculate F(2).
    previous = 1
    current = 1
    # Calculate F(2), F(3), ..., F(n), once each.
    for index in range(2, n + 1):
        # Evaluate both right-hand expressions before rebinding either name.
        previous, current = current, previous + current
    # After the final pass, current stores F(n).
    return current

# Added call makes the slide's stated result visible.
print(fibonacci_iterative(5))
```

Code 2.2: An iterative alternative with the same two base cases.

```
8
```

The simultaneous assignment is particularly important:

`previous, current = current, previous + current.`

Both expressions on the right use the values from *before* the assignment. Python evaluates them before rebinding the names on the left. Replacing the statement with `previous = current` followed by `current = previous + current` would incorrectly use the newly updated value of `previous` in the second statement.

Index	Old previous	Old current	New previous	New current
2	1	1	1	2
3	1	2	2	3
4	2	3	3	5
5	3	5	5	8

Table 2.1.: State trace for `fibonacci_iterative(5)`. The two names store consecutive sequence values, not the same updated value twice.

The iterative version is normally the practical choice for this simple sequence. It avoids recomputing the same terms and does not keep a recursive chain for every stage. The recursive version remains useful for learning how two base cases and two recursive calls work.

Technical clarification

The source's comparison of "many call frames" and "a few variables" concerns active execution state. The total number of calls made over the whole run is different from the maximum number active at once. A branching tree can contain many total calls while only one descending path and its waiting ancestors are active during depth-first evaluation.

2.5. Worked prediction: a different two-base recurrence

Lecture-slide alignment: Week 6 source PDF, p. 42. PDF page numbers include title and divider pages.

Annotated slide example. File: `ex14_sequence.py`.

```python
# The two initial values determine this recurrence.
def sequence(n):
    # First starting value.
    if n == 0:
        return 2
    # Second starting value.
    if n == 1:
        return 3
    # Use both smaller results; assume a nonnegative integer input.
    return 2 * sequence(n - 1) - sequence(n - 2)

# Added calls answer the three predictions in the slide.
for n in [2, 3, 4]:
    print(n, "->", sequence(n))
```

Code 2.3: The two-base-case sequence activity.

```
2 -> 4
3 -> 5
4 -> 6
```

The calculations are

$$S(2) = 2S(1) - S(0) = 2(3) - 2 = 4,$$
$$S(3) = 2S(2) - S(1) = 2(4) - 3 = 5,$$
$$S(4) = 2S(3) - S(2) = 2(5) - 4 = 6.$$

Both base cases are needed because computing $S(2)$ directly invokes arguments 1 and 0. A missing rule for zero would allow the recursion to proceed into negative arguments. A rule for zero alone would not make the second starting value 3 available.

The results suggest $S(n) = n + 2$ for nonnegative integers. Substituting into the recurrence verifies the pattern: $2[(n-1)+2] - [(n-2)+2] = n + 2$, and the two starting values agree. This worked check is derived from the activity; it is not a reason to change the recursive function while tracing it.

2.6. A smaller problem does not have to be a smaller number

Lecture-slide alignment: Week 6 source PDF, pp. 43–44. PDF page numbers include title and divider pages.

The input may be a string, a list, or a nested collection. We need a measurable size and a reduction that preserves the task's meaning. For a string, the size may be its length. For a list, it may be the number of elements still unprocessed. For a finite tree-like structure, it may be the remaining substructure.

Input	Possible reduction	Terminal form
String	Remove its first and last characters.	Empty string or one character.
List	Process the first element and pass the remaining list.	Empty list.
Nested structure	Process a node, then its smaller children.	A node with no relevant children.

The slides mention nested structures as a motivation, but do not implement a general tree traversal. For this week's code, concentrate on strings and ordinary short lists. Reusing the same unchanged object as the recursive input is not a reduction merely because a new function call occurs.

2.7. Palindrome testing as a recursive question

Lecture-slide alignment: Week 6 source PDF, pp. 44–46. PDF page numbers include title and divider pages.

A palindrome reads the same forward and backward. The recursive characterization has two parts: strings of length zero or one are accepted directly; longer strings are accepted only when their first and last characters match *and* the interior substring is itself a palindrome.

Annotated slide example. File: `ex15_palindrome.py`.

```
# Test the supplied characters exactly; no cleaning occurs in this function.
def is_palindrome(text):
    # Cover both odd-length and even-length terminal forms before indexing.
    if len(text) <= 1:
        return True
    # and skips the recursive call when the boundary characters differ.
    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))

# Added calls demonstrate both examples discussed in the slides.
print(is_palindrome("level"))
print(is_palindrome("sensor"))
```

Code 2.4: Test a string's structure using its ends and its interior.

```
True
False
```

The base case must occur before `text[0]` or `text[-1]`. An empty string has no first or last character. A one-character string needs no further comparison because it is already symmetric.

For a longer string, `text[1:-1]` includes the characters beginning at index 1 and stops before the final character. It removes the two boundary characters. For `"level"`, the recursive inputs are

$$\texttt{"level"} \longrightarrow \texttt{"eve"} \longrightarrow \texttt{"v"}.$$

The final call returns `True`. The call with `"eve"` has matching ends and a true interior result, so it returns true. The call with `"level"` does the same. The original string is not modified; slicing supplies a shorter string to another call.

Trace the program

Trace an even-length example as well: `"abba"` becomes `"bb"`, then `""`. There is no one-character stage. This explains why the test is `len(text) <= 1`, not only `len(text) == 1`.

2.8. Short-circuit evaluation avoids unnecessary recursion

Lecture-slide alignment: Week 6 source PDF, pp. 47–48. PDF page numbers include title and divider pages.

The expression `text[0] == text[-1] and is_palindrome(text[1:-1])` is evaluated from left to right. If the end characters do not match, the first condition is false, so the whole conjunction is already false. Python does not evaluate the recursive expression.

For `"sensor"`, the first comparison is `"s" == "r"`, which is false. The function returns immediately; it never inspects the interior `"enso"`. The algorithm has found enough evidence to reject the candidate.

If the boundaries match, the result is not settled. For example, the matching ends of `"abca"` do not prove that the interior `"bc"` is symmetric. The recursive call must still supply that information. Its false result propagates back to the outer call.

Key idea

Short-circuit evaluation can stop unnecessary work, but it does not replace a valid base case. If an empty string reaches the boundary comparison before a length check, indexing fails before `and` can help.

2.9. Separate normalization from structural recursion

Lecture-slide alignment: Week 6 source PDF, pp. 49–50. PDF page numbers include title and divider pages.

The basic palindrome function compares the characters exactly as supplied. Capitalization, spaces, punctuation, and digits therefore matter. The slides add a separate cleaning function when the question is about a human-entered phrase rather than the exact character sequence.

Annotated slide example. File: `ex16_phrase_palindrome.py`.

```python
# Reuse the same structural test, with its base case before indexing.
def is_palindrome(text):
    if len(text) <= 1:  # Empty and singleton strings are terminal.
        return True
    # Compare the ends and recurse only on the shorter interior.
    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))

# Cleaning policy from the slide: lowercase, then retain only letters.
def letters_only_lower(text):
    # Start an empty string to collect accepted characters.
    cleaned = ""
    # lower() returns a transformed string; it does not mutate text.
    for character in text.lower():
        # Digits, spaces, and punctuation fail this letter test.
        if character.isalpha():
            cleaned += character
    # Return the constructed string after inspecting every character.
    return cleaned

# The wrapper connects two functions with different responsibilities.
def is_phrase_palindrome(text):
    # Apply the documented cleaning policy first.
    cleaned = letters_only_lower(text)
    # Ask the recursive structural question about the cleaned string.
    return is_palindrome(cleaned)

# The original phrase is preserved; cleaning produces neveroddoreven.
phrase = "Never odd or even"
print(is_phrase_palindrome(phrase))
```

Code 2.5: Lowercase letter-only cleaning followed by the same recursive test.

```
True
```

The cleaning function starts with an empty result, loops through the lowercase version of the input, and appends each character that passes `isalpha()`. It then returns the accumulated string. The wrapper `is_phrase_palindrome` coordinates cleaning and recursive testing, but does not perform either task itself.

The phrase `"Never odd or even"` becomes `"neveroddoreven"`. The first three recursive levels are

Level	Input	Boundary test and next input
1	`neveroddoreven`	`n == n`; next is `everoddoreve`.
2	`everoddoreve`	`e == e`; next is `veroddorev`.
3	`veroddorev`	`v == v`; next is `eroddore`.

Continuing the process reaches the two-character string `"dd"`, then the empty string. The cleaned phrase has even length, so its terminal form is empty rather than a singleton. Every comparison matches, so the answer is `True`.

Engineering check

Normalization is a declared policy, not a universal improvement. Removing punctuation and digits may be sensible for this phrase exercise but can destroy the identity of a component such as `P-007`. Do not transfer the phrase-cleaning rule into engineering identifiers without checking the data specification.

Technical clarification: what this cleaning rule does and does not promise

`isalpha()` is not restricted to the English alphabet; its behavior follows Python's character rules. The simple code is not a complete language-aware normalization system. It also turns a string containing only spaces, digits, or punctuation into the empty string, which the structural test accepts. Rejecting such input would be an additional policy, not behavior of the given function. The supplied phrase examples use ordinary English letters.

2.10. The first-element plus remaining-list pattern

Lecture-slide alignment: Week 6 source PDF, pp. 51–53. PDF page numbers include title and divider pages.

A short list can be processed by separating its first element from the rest. For summation, the first element contributes its value, and the smaller call contributes the sum of all remaining elements. The empty list contributes zero.

Annotated slide example. File: `ex17_recursive_sum.py`.

```
# Assume a short list of compatible numerical values in one unit.
def recursive_sum(values):
    # The empty remainder contributes the additive identity.
    if len(values) == 0:
        return 0
    # Add the first value to the sum of the shorter remaining list.
    return values[0] + recursive_sum(values[1:])

# Preserve the exact numerical dataset used in the slides.
readings = [2.0, 3.5, 4.0]
# No physical unit is specified for these three slide values.
print(recursive_sum(readings))
```

Code 2.6: Recursive summation of the slide's three numerical readings.

```
9.5
```

The expansion is

$$\begin{aligned}\text{sum}([2.0, 3.5, 4.0]) &= 2.0 + \text{sum}([3.5, 4.0]) \\ &= 2.0 + 3.5 + \text{sum}([4.0]) \\ &= 2.0 + 3.5 + 4.0 + \text{sum}([]) \\ &= 2.0 + 3.5 + 4.0 + 0 = 9.5.\end{aligned}$$

Here sum in the handwritten expansion denotes the recursive operation, not a call to Python's built-in `sum`. Each nonterminal call receives a list one element shorter. The base result is the additive identity. The unit of the result is the common unit of the input quantities; the source slide does not specify a physical unit for these three demonstration numbers.

2.10.1. The sentinel question in the slide

If the list contains a sensor error code such as -999, the recursive sum still treats it as a number. For example, $2 + (-999) + 4 = -993$ is the correct arithmetic sum of those values but may be a meaningless measurement summary. The recurrence does not know what the error code means.

Choose and document a data policy before calling the algorithm. One possible policy is to reject a record containing a known missing-value code. Another is to create a separate filtered list and explicitly report which observations were excluded. The slide raises this question but does not select a universal policy. Negative readings are not automatically invalid for every measured quantity.

Technical clarification

The slice `values[1:]` creates a new outer list. These examples intentionally use short lists so the reduction is visible. Repeated slicing also does copying work; it is not a memory-free operation. For a large ordinary numerical sum, the source's preference for an iterative or standard accumulation approach remains relevant.

2.11. Recursive search with two stopping cases

Lecture-slide alignment: Week 6 source PDF, pp. 54–55. PDF page numbers include title and divider pages.

A search can finish because the target is found or because no candidates remain. Both are direct answers, but they mean different things.

Annotated slide example. File: `ex18_contains.py`.

```python
# Return whether target occurs; matching uses exact equality.
def contains_recursive(values, target):
    # Base case 1: the search space has been exhausted.
    if len(values) == 0:
        return False
    # Base case 2: a match already provides sufficient evidence.
    if values[0] == target:
        return True
    # Keep the target fixed but shorten the remaining search space.
    return contains_recursive(values[1:], target)

# Added calls illustrate success, failure, and an empty input.
print(contains_recursive([2.0, 3.5, 4.0], 3.5))
print(contains_recursive([2.0, 3.5, 4.0], 9.0))
print(contains_recursive([], 3.5))
```

Code 2.7: A recursive exact-equality membership search.

```
True
False
False
```

For the successful example, the first call compares 2.0 with 3.5 and continues. The second call sees 3.5 at the front and returns `True`. The outer call returns that same Boolean. The remaining value 4.0 is not inspected because the existence question is already answered.

For target 9.0, none of the three values matches. The calls receive lists of lengths 3, 2, 1, and 0.

The empty-list call returns `False`, and that false result passes back through every waiting frame. The target remains unchanged throughout; only the unsearched list becomes smaller.

The exact `==` test is appropriate for exact identifiers or deliberately exact numerical cases. A search for a measured floating-point value may instead need a declared tolerance, but the source function does not implement one. Do not describe its behavior as approximate matching.

Engineering check

The same early-stopping idea applies to monitoring: one threshold violation can be sufficient to answer "Does any violation exist?" However, answering "How many violations occurred?" requires continuing through the remaining readings. Define the output question before choosing an early return.

2.12. Debug the non-numerical recursion

Lecture-slide alignment: Week 6 source PDF, pp. 56–57. PDF page numbers include title and divider pages.

The first faulty palindrome function sends the entire original string to the next call.

Annotated slide example. File: `ex19_palindrome_no_shrink.py`.

```
# DELIBERATE ERROR: the recursive call gets the original string again.
def is_palindrome(text):
    # This base case cannot be reached from a longer matching string.
    if len(text) <= 1:
        return True
    # Wrong argument: text should be text[1:-1].
    return (text[0] == text[-1]
            and is_palindrome(text))

# Matching end characters expose the defect; sensor would return False.
print(is_palindrome("level"))
```

Code 2.8: Deliberate error: a recursive string input with unchanged length.

For matching ends such as those of `"level"`, the same comparison repeats until Python raises `RecursionError`. A test with `"sensor"` alone could miss the defect: its unequal ends cause immediate short-circuit rejection. Tests must exercise the recursive path, not only a terminal rejection path.

The second faulty function accepts only a one-character base case.

Annotated slide example. File: `ex20_palindrome_narrow.py`.

```python
# DELIBERATE ERROR: only the singleton terminal form is handled.
def is_palindrome(text):
    # An even-length palindrome eventually has length zero instead.
    if len(text) == 1:
        return True
    # On the empty string, text[0] raises IndexError.
    return (text[0] == text[-1]
            and is_palindrome(text[1:-1]))

# The chain abba -> bb -> empty reveals the missing base case.
print(is_palindrome("abba"))
```

Code 2.9: Deliberate error: a terminal structure was omitted.

An even-length palindrome such as `"abba"` becomes empty after repeated interior slicing. With no zero-length base case, the code tries `text[0]` on that empty string and raises `IndexError`. The remedy is `len(text) <= 1`, placed before the indexing expression.

Answer and explanation

These defects have different diagnoses. In the first, the size measure never decreases along an attempted recursive path. In the second, it decreases correctly but arrives at a terminal input that the function fails to handle. A sound explanation identifies both the reduction rule and the set of possible terminal forms.

2.13. A general recipe and recursive reversal

Lecture-slide alignment: Week 6 source PDF, pp. 58–61. PDF page numbers include title and divider pages.

To design non-numerical recursion, define the structure, choose a size measure, identify the smallest directly answerable structures, process one part, create a smaller input, and combine the current evidence with the returned sub-result. Test empty, singleton, short normal, and malformed inputs against the declared contract.

For reversing a string, the base cases are an empty string and a singleton. A smaller input is the suffix after the first character. Reverse that suffix, then append the original first character at the end.

Annotated slide example. File: `ex21_reverse.py`.

```python
# Return a new reversed string; never mutate the input.
def reverse_recursive(text):
    # Empty and one-character strings are already reversed.
    if len(text) <= 1:
        return text
    # Reverse the tail first, then put the current first character last.
    return reverse_recursive(text[1:]) + text[0]

# The slide example reverses the two-character university abbreviation.
print(reverse_recursive("NU"))
```

Code 2.10: One possible recursive string reversal from the slides.

```
UN
```

For "NU", the function calls itself with "U"; that base case returns "U". The original call then evaluates "U" + "N", producing "UN". For "ABC", the pending combinations are

$$\text{reverse}(\texttt{"BC"}) + \texttt{"A"} \quad \text{and} \quad \text{reverse}(\texttt{"C"}) + \texttt{"B"}.$$

The returns are "C", then "CB", then "CBA". Reversing the order of concatenation would change the result: placing text[0] first simply reconstructs the original order rather than reversing it.

Key idea

For every recursive function, explain four facts aloud: its base cases, its smaller inputs, its termination argument, and its combination rule. Saying "it works because it calls itself" leaves all four unanswered.

2.14. Lecture 17 exit ticket: count a letter

Lecture-slide alignment: Week 6 source PDF, p. 62. PDF page numbers include title and divider pages.

The prompt asks for a recursive function that counts occurrences of a letter. The empty string should return zero. The first character contributes one if it matches and zero otherwise. The recursive input is text[1:], and the combination is "current contribution plus the count in the remaining suffix."

Worked slide activity solution. File: ex22_exit17.py.

```python
# Worked solution; target is exactly one character and matching is case-sensitive.
def count_letter(text, target):
    # An empty string contains zero occurrences of any single character.
    if len(text) == 0:
        return 0
    # Convert the current character's match into a contribution of 0 or 1.
    if text[0] == target:
        first_count = 1
    else:
        first_count = 0
    # Add the contribution to the count in the strictly shorter suffix.
    return first_count + count_letter(text[1:], target)

# Boundary and normal tests developed from the exit-ticket prompt.
print(count_letter("", "a"))
print(count_letter("banana", "a"))
```

Code 2.11: A worked solution to the recursive character-counting exit ticket.

```
0
3
```

For "banana" and target "a", the contributions are $0, 1, 0, 1, 0, 1$, followed by the empty-string base result zero. Their sum is 3. The input contract here is a string and a *single* target character, with case-sensitive matching. Passing "an" would ask a different substring problem that this one-character comparison does not implement.

3 Dictionaries and Assignment 3

Lecture-slide alignment: Lecture 18, source PDF pp. 63–93.

Learning outcomes

After this chapter, you should be able to create and access dictionaries, add and update entries, handle absent keys, use dictionary views and iteration, represent nested material and sensor records, evaluate design dictionaries, and explain a consistent key and unit convention.

3.1. Warm-up: what does position 2 mean?

Lecture-slide alignment: Week 6 source PDF, pp. 63–66. PDF page numbers include title and divider pages.

The slides begin with the record `sensor = ["T-101", "C", 80.0, True]`. The values themselves are not difficult to store. The difficulty is their interpretation: a reader must remember that position 0 is an identifier, position 1 is a unit, position 2 is a warning limit, and position 3 is a calibration flag. Inserting a new field can change those positions and silently break code that assumes the old layout.

A **dictionary** stores relationships between keys and values. It is a **mapping**: a key identifies an associated value. Instead of asking for the third item, the program asks for the value named `"warning_limit"`. The mapping makes each field's meaning more visible at the point of use.

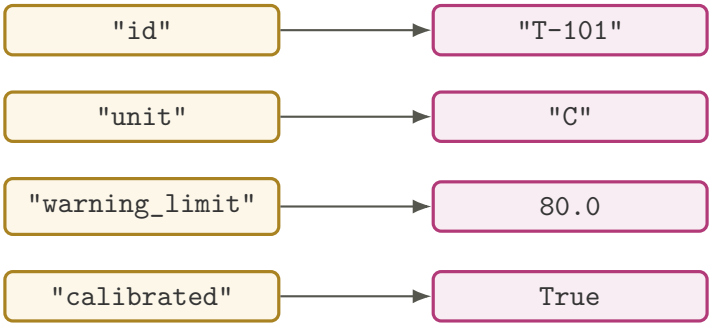


Figure 3.1.: The key–value relationships from source PDF p. 66. Each key names one field, while the values may have different Python types.

A dictionary does not replace all lists. A measurement series still needs ordered repeated values. A record of metadata needs named fields. Often the best representation combines a dictionary for the record with a list for its repeated readings.

3.2. Create and inspect the first dictionary

Lecture-slide alignment: Week 6 source PDF, pp. 67–68. PDF page numbers include title and divider pages.

Annotated slide example. File: `ex23_dictionary.py`.

```python
# Store different kinds of sensor information under meaningful keys.
sensor = {
    "id": "T-101",          # A text identifier, not an integer position.
    "unit": "C",            # Temperature scale used by the limit below.
    "warning_limit": 80.0,  # Limit in degrees Celsius in this example.
    "calibrated": True      # A Boolean flag, not a calibration date.
}
# Display the record and identify its Python type.
print(sensor)
print(type(sensor))
# Square brackets retrieve by key rather than by numerical position.
sensor_id = sensor["id"]
limit_c = sensor["warning_limit"]
# These two outputs correspond to the retrieval slide.
print(sensor_id)
print(limit_c)
```

Code 3.1: Create the source sensor record and retrieve two named fields.

```
{'id': 'T-101', 'unit': 'C', 'warning_limit': 80.0, 'calibrated': True}
<class 'dict'>
T-101
80.0
```

The outer curly braces begin and end the dictionary literal. Within them, each entry has the form `key: value`. The colon associates the key on the left with the value on the right; commas separate entries. Quotation marks make keys such as `"id"` and `"unit"` strings. The values can be strings, numbers, Booleans, or other appropriate objects.

`sensor["id"]` retrieves the value associated with the exact key `"id"`. It does not search the values for the word `id`. The expression `sensor["warning_limit"]` returns the floating-point value 80.0, which the code assigns to `limit_c`.

The generic source key `"warning_limit"` acquires its unit from the record's `"unit": "C"` field. The notes retain this schema. A later application can instead adopt a unit-bearing key such as `"warning_limit_c"`, but all code using the record must then adopt the same convention.

Stop, predict, and try it

Predict the type of each retrieved field in this example. The identifier is a string, the warning limit is a floating-point number, and the calibration flag is a Boolean. A dictionary can be useful precisely because its fields have different meanings and types.

3.3. Keys, values, and a consistent vocabulary

Lecture-slide alignment: Week 6 source PDF, p. 69. PDF page numbers include title and divider pages.

The source lists strings, integers, and suitable tuples as common key choices. It allows many types of values, including numbers, strings, Booleans, lists, tuples, dictionaries, and functions. A value need not have the same type as the value of the next field.

Within one dictionary, each key identifies one current value. Updating the same key replaces that association instead of creating a second independent entry under the same key. Different

keys may legitimately share the same value: two sensors can both have unit `"C"`, for example.

Technical clarification: the precise Python key requirement

The technical requirement is that a dictionary key be *hashable*. Strings, integers, and tuples whose elements are hashable meet this requirement; ordinary lists and dictionaries do not. Hashing is not taught this week, but the distinction prevents the overgeneralization that every tuple is a valid key. The supplementary example in Appendix A also shows that an integer key is a key, not an element position. See the Python mapping reference [6].

A **schema** is the agreed structure of a record: its key names, expected value types, required and optional fields, and unit conventions. For example, `"yield_strength"`, `"YieldStrength"`, and `"strength"` are three distinct string keys. A human may see a similar meaning; Python sees different strings. Choose a vocabulary deliberately and use it consistently.

Design principle

A good key answers a stable question. Prefer `"yield_strength_pa"` to a vague key such as `"value"` when the record contains several physical properties. Meaningful keys reduce guesswork, but do not automatically validate the quantity, its unit, or its provenance.

3.4. Add or update an entry

Lecture-slide alignment: Week 6 source PDF, p. 70. PDF page numbers include title and divider pages.

The same assignment syntax either adds a new key or replaces the value of an existing one.

Annotated slide example. File: `ex24_add_update.py`.

```
# Classroom example values; all densities are in kg/m^3.
material_density = {
    "steel": 7850,      # Original steel entry.
    "aluminum": 2700    # Original aluminum entry.
}
# A previously absent key adds a new entry.
material_density["copper"] = 8960
# An existing key replaces its value; this is a what-if edit, not new evidence.
material_density["aluminum"] = 2710
# The dictionary has three entries, not two aluminum entries.
print(material_density)
```

Code 3.2: Adding copper and updating aluminum in the source's density example.

```
{'steel': 7850, 'aluminum': 2710, 'copper': 8960}
```

The initial dictionary has two entries. Adding `"copper"` increases that to three. Updating `"aluminum"` leaves the number of keys unchanged; the new value 2710 replaces the previous 2700. The assignment mutates the existing dictionary object.

Engineering check

The change from 2700 to 2710 demonstrates an update operation; the slide supplies no experimental justification or source establishing that one is a correction to the other. Treat it as a classroom what-if edit. Do not overwrite a reference database just because assignment syntax makes it easy.

The same aliasing issue studied with lists applies to dictionaries: assigning a second name does not make a copy. A top-level `copy()` can separate later top-level edits, but nested mutable values may still be shared. Appendix A illustrates that limitation using a nested list of sensor readings, in support of the slide's warning about unexpected mutation.

3.5. Direct access and the missing-key error

Lecture-slide alignment: Week 6 source PDF, p. 71. PDF page numbers include title and divider pages.

A direct lookup assumes the requested key exists. If it does not, Python raises `KeyError`.

Annotated slide example. File: `ex25_key_error.py`.

```
# DELIBERATE ERROR: this example dictionary has no titanium entry.
material_density = {
    "steel": 7850,      # Density in kg/m^3.
    "aluminum": 2700    # Density in kg/m^3.
}
# Direct access raises KeyError rather than inventing a density.
print(material_density["titanium"])
```

Code 3.3: Deliberate error: titanium is absent from the dictionary.

The final exception is `KeyError: 'titanium'`. This is not a density of zero and not a negative answer to a materials question. It is a statement about the data representation: the requested key was absent at the time of lookup.

Possible causes include a misspelling, an unexpected space, a different case convention, an unsupported material, or an incomplete database. Diagnose which assumption failed before choosing a remedy. Automatically creating a numerical default may hide the error and contaminate later calculations.

3.6. Test membership before retrieval

Lecture-slide alignment: Week 6 source PDF, p. 72. PDF page numbers include title and divider pages.

When absence is a normal possibility, check membership and handle both paths explicitly.

Annotated slide example. File: `ex26_membership.py`.

```python
# Recreate the two-entry slide dataset so this example runs independently.
material_density = {"steel": 7850, "aluminum": 2700}
# This key is intentionally absent.
selected_material = "titanium"
# For a dictionary, in tests keys, not stored values.
if selected_material in material_density:
    # Direct access is now justified by the membership test.
    density = material_density[selected_material]
    print(f"Density = {density} kg/m^3")
else:
    # Make the missing-data path visible to the user.
    print("Material not found in database")
```

Code 3.4: A membership guard around direct access.

```
Material not found in database
```

For a dictionary `d`, the expression `key in d` checks keys. It does not search the stored values. Thus `"steel" in material_density` asks whether a steel entry exists. The expression `7850 in material_density` asks whether the integer 7850 is a key, not whether a density of 7850 occurs.

If a value-only search is really intended, the code must say so through the values view. More often, the name–value association is important, so use a key lookup or iterate over pairs rather than separating the two pieces of information prematurely.

Key idea

A guarded lookup makes the decision visible: if the key exists, retrieve and use the value; otherwise report the missing information. The absence of a key can be a legitimate result that the program should communicate, not an event to conceal.

3.7. Use get when a default has a clear meaning

Lecture-slide alignment: Week 6 source PDF, p. 73. PDF page numbers include title and divider pages.

`d.get(key, default)` returns the stored value when the key exists and the supplied default otherwise. If the default is omitted, it is `None`. Unlike direct indexing, a missing key does not itself cause `KeyError` in this operation.

Annotated slide example. File: `ex27_get.py`.

```python
# All recorded densities are known numbers in kg/m^3.
material_density = {"steel": 7850, "aluminum": 2700}
# Return None when the requested key is absent; do not add the key.
density = material_density.get("titanium", None)
# Test the sentinel before treating density as a number.
if density is None:
    print("Material not found")
else:
    # Print a physical unit only when a stored numerical value exists.
    print(f"Density = {density} kg/m^3")
```

Code 3.5: A visible missing-data path using a sentinel.

```
Material not found
```

A **sentinel** is a distinguished value that represents a special condition rather than an ordinary measurement. Here, `None` means that no material density was found. The caller tests `density is None` before using `density` numerically. This maintains the difference between "absent" and "measured or specified as zero."

Common mistake

`material_density.get("titanium", 0)` can make a later mass calculation appear to succeed while using a fabricated zero density. Avoid a physically meaningful number as a missing-data substitute unless that behavior is explicitly justified by the model and data policy.

A default is returned, not inserted. After an unsuccessful `get`, the dictionary still lacks the requested key. Also, if a real stored value is already `None`, this simple check cannot distinguish "present but unknown" from "key absent." A membership test can distinguish the two when the schema requires it. For the source density examples, all recorded values are numbers, so `None` unambiguously signals a failed lookup.

3.8. Keys, values, items, and iteration

Lecture-slide alignment: Week 6 source PDF, pp. 74–75. PDF page numbers include title and divider pages.

Dictionary operations offer three useful views. `keys()` exposes the identifiers; `values()` exposes the associated values; `items()` exposes the key–value pairs. Direct iteration over a dictionary yields its keys.

Annotated slide example. File: `ex28_views.py`.

```
# A fresh dictionary fixes the starting state for this demonstration.
material_density = {"steel": 7850, "aluminum": 2700}
# Each method returns a view, not a plain list.
print(material_density.keys())
print(material_density.values())
print(material_density.items())
# Direct dictionary iteration yields one key at a time.
for material in material_density:
    # Use the current key to retrieve the corresponding density.
    density = material_density[material]
    # This display retains the association between name and value.
    print(material, density)
```

Code 3.6: Inspect the three views and loop over keys.

```
dict_keys(['steel', 'aluminum'])
dict_values([7850, 2700])
dict_items([('steel', 7850), ('aluminum', 2700)])
steel 7850
aluminum 2700
```

The printed representations `dict_keys(...)`, `dict_values(...)`, and `dict_items(...)` are not ordinary list literals. The word *view* is meaningful: these objects reflect the underlying dictionary. Appendix A compares a live view with a separately created list snapshot.

3.8.1. Choose the iteration form by the task

Use keys when the names are all that is needed, values for genuine value-only processing, and items when each label should stay paired with its value. The last form usually makes a named engineering report clearer.

Annotated slide example. File: `ex29_items.py`.

```
# Integer densities match the d formatting code used below.
material_density = {"steel": 7850, "aluminum": 2700}
# Headings communicate the purpose of the following rows.
print("MATERIAL DENSITY TABLE")
print("----------------------")
# Unpack each key-value pair into meaningful local names.
for material, density in material_density.items():
    # 10s formats text; 5d formats an integer; include the physical unit.
    print(f"{material:10s}: {density:5d} kg/m^3")
```

Code 3.7: Unpack key–value pairs into a formatted density table.

```
MATERIAL DENSITY TABLE
----------------------
steel     :  7850 kg/m^3
aluminum  :  2700 kg/m^3
```

In `for material, density in material_density.items()`, each iteration unpacks a pair into two names. This builds directly on Week 5 tuple unpacking. The formatted field `:10s` allocates a minimum width of ten characters for text; `:5d` formats an integer in a minimum width of five. The supplied densities are integers. A density stored as a `float` would require a compatible

floating-point format instead of `d`.

Technical clarification

Dictionaries preserve insertion order in current Python, but key access is not positional access. The printed order of a table should not be mistaken for a sorted material ranking. The key-based meaning remains the same regardless of the order in which the entries were inserted. The source does not require a sorting algorithm for this table.

3.9. Remove entries deliberately

Lecture-slide alignment: Week 6 source PDF, p. 76. PDF page numbers include title and divider pages.

`del d[key]` deletes a known entry. It raises `KeyError` if the key is absent. `d.pop(key, default)` removes an existing entry and returns its value; when the key is absent it returns the supplied default. Without a default, a missing key also raises `KeyError` for `pop`.

Annotated slide example. File: `ex30_remove.py`.

```
# First alternative: use del when the key is known to exist.
material_density = {"steel": 7850, "copper": 8960}
# This statement deletes the copper entry; it has no returned value.
del material_density["copper"]
# Added output reveals the state after deletion.
print(material_density)

# Reset before trying the second alternative independently.
material_density = {"steel": 7850, "copper": 8960}
# pop deletes the entry and returns its value, or None if absent.
removed = material_density.pop("copper", None)
# Retain both the removed value and evidence of the remaining data.
print(removed)
print(material_density)
```

Code 3.8: Deletion and removal-with-return shown as separate alternatives.

```
{'steel': 7850}
8960
{'steel': 7850}
```

The reset between the two alternatives is intentional. Deleting copper and then immediately trying to demonstrate its removal again would not start from the same state. When comparing operations, rebuild the starting dictionary or explicitly document the state left by earlier statements.

Engineering check

Removal is a change to evidence. Preserve the source data when required and document why a property or measurement was removed. Keeping the removed value can be useful for an audit trail, but `pop` alone does not record the reason, author, or time of the change.

3.10. An interactive material-density lookup

Lecture-slide alignment: Week 6 source PDF, p. 77. PDF page numbers include title and divider pages.

The complete lookup combines several earlier concepts: text input, normalization, dictionary access, a sentinel check, and a unit-bearing output.

Annotated slide example. File: `ex31_material_input.py`.

```
# Preserve the three classroom values and their common unit.
material_density_kg_m3 = {
    "steel": 7850,
    "aluminum": 2700,
    "copper": 8960
}
# Normalize a human-entered name, not a case-sensitive technical identifier.
selected = input("Material: ").strip().lower()
# With no explicit default, get returns None for a missing key.
density = material_density_kg_m3.get(selected)
# Never convert an absent material into a fictitious zero density.
if density is None:
    print("Unknown material")
else:
    # A successful lookup reports the stored value and its unit.
    print(f"Density = {density} kg/m^3")
```

Code 3.9: Read a material name and report its stored density.

```
Material:  Steel
Density = 7850 kg/m^3
```

The displayed transcript uses the input `" Steel "`. `strip()` removes surrounding whitespace, and `lower()` turns the remaining word into `"steel"`, which matches the stored key. Unknown names follow the visible failure path instead of receiving an invented property value.

Function contract

The lookup vocabulary is exactly `steel`, `aluminum`, and `copper`, after the stated case and whitespace normalization. The three numbers are the lecture's illustrative density values in kg/m^3. The slides do not identify alloy grades, temperature, measurement uncertainty, or authoritative property sources; this dictionary is a programming example, not a certified materials database.

Test the successful path with a stored name, the normalization path with spaces and capitals, and the absent-key path with an unsupported name such as `titanium`. A good report should reveal which path occurred and what the result means.

3.11. Nested dictionaries for related properties

Lecture-slide alignment: Week 6 source PDF, pp. 78–79. PDF page numbers include title and divider pages.

A material has more than one property. A nested dictionary can map each material name to an inner dictionary that maps property names to their values. Two successive lookups therefore select the material and then the property.

Annotated slide example. File: `ex32_nested.py`.

```python
# These are the slide's illustrative values, not certified design data.
materials = {
    "steel": {  # Outer key selects one complete material record.
        "density_kg_m3": 7850,       # kg/m^3.
        "yield_strength_pa": 250e6,  # Pa, equivalent to 250 MPa.
        "elastic_modulus_pa": 200e9  # Pa, equivalent to 200 GPa.
    },
    "aluminum": {
        "density_kg_m3": 2700,       # kg/m^3.
        "yield_strength_pa": 150e6,  # Pa, equivalent to 150 MPa.
        "elastic_modulus_pa": 69e9   # Pa, equivalent to 69 GPa.
    }
}
# Select the steel record first, then the required property in that record.
steel_strength_pa = materials["steel"]["yield_strength_pa"]
# Divide by 1e6 only for display in MPa; retain the stored value in Pa.
print(f"Steel yield strength = "
      f"{steel_strength_pa / 1e6:.1f} MPa")
```

Code 3.10: The nested property records and a two-stage retrieval path.

```
Steel yield strength = 250.0 MPa
```

Read `materials["steel"]["yield_strength_pa"]` in two stages. First select the dictionary stored under `"steel"`. Then select its `"yield_strength_pa"` entry. The first lookup returns another dictionary, while the second returns a number.

The keys include units: density in kg/m^3, strength in Pa, and modulus in Pa. The printed report divides the strength by 10^6 to display MPa. That conversion does not change the value stored in the original record. It changes only the expression supplied for output.

Selection	Meaning	Result type
`materials`	All material records.	`dict`
`materials["steel"]`	All stored steel properties.	`dict`
`["yield_strength_pa"]` on the inner record	One requested property.	`float`

3.11.1. Two levels mean two contracts

Finding the outer key does not prove that every requested inner property exists. The source's nested dataset supplies a consistent complete set of keys for each material. Code built for partially populated records must validate the required inner fields too. Do not assume that a successful first lookup guarantees every deeper lookup.

3.12. Normalize the outer name, then inspect the record

Lecture-slide alignment: Week 6 source PDF, p. 80. PDF page numbers include title and divider pages.

Annotated slide example. File: `ex33_nested_input.py`.

```python
# Recreate the source data so the lookup is independently runnable.
materials = {
    "steel": {"density_kg_m3": 7850,
              "yield_strength_pa": 250e6,
              "elastic_modulus_pa": 200e9},
    "aluminum": {"density_kg_m3": 2700,
                 "yield_strength_pa": 150e6,
                 "elastic_modulus_pa": 69e9}
}
# Match the capitalization and whitespace convention of the outer keys.
selected = input("Enter material name: ").strip().lower()
# Test the outer key before retrieving its nested record.
if selected in materials:
    record = materials[selected]  # This is the stored inner dictionary.
    density = record["density_kg_m3"]  # Required property in kg/m^3.
    strength_mpa = record["yield_strength_pa"] / 1e6  # Pa -> MPa.
    # Report the selected name and the interpretation of each number.
    print(f"Material: {selected}")
    print(f"Density: {density} kg/m^3")
    print(f"Yield strength: {strength_mpa:.1f} MPa")
else:
    # Lowercasing does not turn aluminium into the key aluminum.
    print("Unknown material")
```

Code 3.11: A complete nested lookup with an explicit unknown-material branch.

```
Enter material name: aluminum
Material: aluminum
Density: 2700 kg/m^3
Yield strength: 150.0 MPa
```

Using the intermediate name `record` reduces visual complexity. Each later expression clearly asks for a property of the selected material. It also makes debugging easier: inspect `selected`, then `record`, then the individual property rather than interpreting a long nested expression all at once.

Common mistake

`"aluminum"` and `"Aluminium"` are not the same key. Lowercasing the second produces `"aluminium"`, not `"aluminum"`. Case and whitespace normalization do not resolve different spellings or synonyms. The source highlights this limitation; a controlled alias vocabulary would be an additional design choice.

3.13. Sensor metadata and repeated readings

Lecture-slide alignment: Week 6 source PDF, pp. 81–82. PDF page numbers include title and divider pages.

The temperature-sensor example combines a dictionary of metadata with a list of measured values. Metadata describe the measurement: identifier, quantity, unit, location, calibration flag, and warning limit. The list contains repeated observations of the same quantity in their original order.

Annotated slide example. File: `ex34_sensor_readings.py`.

```python
# A dictionary holds fields with different meanings and different types.
sensor = {
    "id": "T-101",              # Stable sensor identifier.
    "quantity": "temperature",  # Quantity measured.
    "unit": "C",                # Celsius scale for values and limit.
    "location": "heat exchanger outlet",  # Measurement location.
    "calibrated": True,          # Status flag supplied by the example.
    "warning_limit": 80.0        # Strict exceedance is tested below.
}
# A list stores repeated values of the same quantity in acquisition order.
sensor["readings_c"] = [72.1, 73.5, 75.0, 81.2, 79.8]
# Retrieve the limit once before scanning the measurements.
limit_c = sensor["warning_limit"]
for reading_c in sensor["readings_c"]:
    # Equality at 80.0 does not trigger this strictly-greater-than rule.
    if reading_c > limit_c:
        # Only 81.2 triggers a warning in the supplied dataset.
        print(f"Warning: {reading_c:.1f} C")
```

Code 3.12: A named sensor record containing the slide's five readings.

```
Warning: 81.2 C
```

The five readings are 72.1, 73.5, 75.0, 81.2, and 79.8 degrees Celsius. The rule tests `reading_c > limit_c` against 80.0. Only 81.2 exceeds the threshold, so only that value is printed as a warning. A hypothetical reading exactly equal to 80.0 would not trigger this particular rule.

The field `"calibrated": True` is a supplied Boolean flag. It is not evidence of a calibration date, uncertainty, or traceable certificate. These additional fields would need real supporting information if the record were used beyond a teaching example.

Engineering check

A dictionary keeps units visible, but Python does not enforce unit compatibility. If the readings are in Celsius and the limit is accidentally entered in Kelvin, the comparisons still execute but answer the wrong physical question. The field name, unit field, comments, and report must agree.

3.13.1. Why a list remains appropriate inside the record

A dictionary labels the fields of one sensor. The list records the sequence of readings for that sensor. Replacing the list with unrelated keys such as `"reading1"`, `"reading2"`, and so on would make general iteration awkward and shift the burden to manual naming. Use a structure that matches the repeated nature of the data.

3.14. Evaluate a list of design dictionaries

Lecture-slide alignment: Week 6 source PDF, pp. 83–84. PDF page numbers include title and divider pages.

A list of dictionaries is useful when there are many records with the same schema. The list identifies the sequence of candidate designs, while each dictionary names its design's fields.

Annotated slide example. File: `ex35_designs.py`.

```python
# Preserve the slide's design data; the cost unit is not specified.
designs = [
    {"name": "A", "area_mm2": 300, "cost": 12.5},
    {"name": "B", "area_mm2": 450, "cost": 16.0},
    {"name": "C", "area_mm2": 600, "cost": 20.0}
]
# This simplified axial model uses a common load and stress criterion.
force_n = 12000.0
allowable_stress_pa = 80e6  # Pa; this is 80 MPa.

# The list length stays fixed; each contained dictionary is updated.
for design in designs:
    # Convert square millimetres to square metres.
    area_m2 = design["area_mm2"] / 1_000_000
    # Calculate uniform axial stress in Pa from N and m^2.
    stress_pa = force_n / area_m2
    # Add calculated evidence to the current design record.
    design["stress_mpa"] = stress_pa / 1e6
    # Retain the slide labels: safe means passes this one stated rule only.
    design["status"] = (
        "safe" if stress_pa <= allowable_stress_pa else "unsafe")

# Keep the reporting loop separate from the calculation loop.
print("DESIGN   AREA (mm^2)   STRESS (MPa)   STATUS")
for design in designs:
    # Adjacent f-strings form one formatted row; widths align the columns.
    print(f"{design['name']:>6s}"
          f"{design['area_mm2']:14.0f}"
          f"{design['stress_mpa']:15.1f}"
          f"{design['status']:>10s}")
```

Code 3.13: Compute stress in each design record, then print a separate table.

```
DESIGN   AREA (mm^2)   STRESS (MPa)   STATUS
     A            300           40.0     safe
     B            450           26.7     safe
     C            600           20.0     safe
```

3.14.1. Trace the unit conversion

For design A, the area is $300\,\text{mm}^2$. Because one square metre contains 10^6 square millimetres,

$$A = 300/10^6 = 0.0003\ \text{m}^2, \qquad \sigma = \frac{12000}{0.0003} = 40 \times 10^6\ \text{Pa} = 40\ \text{MPa}.$$

The same calculation gives approximately 26.6667 MPa for design B and 20 MPa for design C. All are below the supplied 80 MPa criterion.

3.14.2. What is being mutated?

During `for design in designs`, the name `design` refers to the current dictionary in the list. Assigning `design["stress_mpa"]` and `design["status"]` adds fields to that existing dictionary. These fields remain available in the later reporting loop. The code is not creating a new isolated dictionary at every pass.

This operation does not change the length of the outer list, so it is different from deleting list elements while iterating directly over that list. Nevertheless, it does change the original candidate records. A real workflow that must preserve the original records should create suitable working copies or separate result records.

3.14.3. Interpret the status labels carefully

Engineering meaning of the slide labels

The source uses `"safe"` and `"unsafe"`. These labels are retained in the example. Here `"safe"` means only "passes the supplied uniform-axial-stress comparison with 80 MPa." It does not certify a component against other load cases, buckling, fatigue, connections, manufacturing defects, or a real design standard. The slides specify no currency for `cost`, so these notes do not assign one or claim a cost optimum.

The conditional expression `"safe" if condition else "unsafe"` chooses one string value. It has the same role as a two-branch assignment. The calculation loop stores evidence; the reporting loop determines display widths, decimal places, and labels. Separating these responsibilities keeps calculation and communication understandable.

3.15. Choose among a list, tuple, and dictionary

Lecture-slide alignment: Week 6 source PDF, p. 85. PDF page numbers include title and divider pages.

Question	List	Tuple	Dictionary
Access	By position.	By position.	By key.
Change	Elements can be added, removed, or replaced.	No item assignment.	Entries can be added, removed, or updated.
Typical role	Repeated ordered values.	A short fixed ordered record.	Named fields or lookup relationships.
Example	Sensor time series.	An (x, y, z) coordinate.	Sensor metadata.
Main beginner risk	Index errors and aliases.	Unclear positional meaning.	Missing or inconsistent keys.

No structure automatically gives a record physical meaning. A tuple of three numbers needs a documented coordinate convention. A list of measurements needs units and order. A dictionary needs a stable schema and a missing-data policy. A nested combination is often more expressive than forcing every part into one collection type.

3.16. Debug a failed nested lookup

Lecture-slide alignment: Week 6 source PDF, pp. 86–87. PDF page numbers include title and divider pages.

The activity contains a mismatch before the numerical property is ever retrieved.

Annotated slide example. File: `ex36_bad_nested_lookup.py`.

```
# DELIBERATE ERROR: lookup conventions do not match the supplied text.
properties = {
    "steel": {"density": 7850},     # Density values in kg/m^3.
    "aluminum": {"density": 2700}
}
# Whitespace and case make this different from the key steel.
material = " Steel "
# Missing lookup returns None; get does not repair the key automatically.
record = properties.get(material)
# None is not a dictionary, so this raises TypeError, not KeyError.
print(record["density"])
```

Code 3.14: Deliberate error: normalization and a sentinel check are missing.

The earliest violated assumption is that `material` matches the outer key convention. It contains spaces and an uppercase initial, whereas the dictionary uses lowercase keys without surrounding whitespace. The `get` call therefore returns `None`. The next expression attempts to subscript that `None`, raising `TypeError: 'NoneType' object is not subscriptable`. This particular failure is *not* a `KeyError`.

Worked slide activity solution. File: `ex37_repaired_lookup.py`.

```
# Worked repair: keep the source data and missing-data meaning intact.
properties = {
    "steel": {"density": 7850},
    "aluminum": {"density": 2700}
}
# Normalize the human-entered material vocabulary before lookup.
material = " Steel ".strip().lower()
# The optional return is a dictionary or None.
record = properties.get(material)
# Check the outer result before attempting nested indexing.
if record is None:
    print("Unknown material")
else:
    # Here the schema promises a density field, measured in kg/m^3.
    print(record["density"])
```

Code 3.15: Worked repair that keeps absence visible.

```
7850
```

The repair first normalizes the human-entered name, then checks whether the outer lookup returned a record. It does not supply a fake density. The simple inner schema promises a `"density"` field. A more variable dataset would also require handling a missing inner property.

Pattern	Likely symptom	First useful check
Missing key	`KeyError` on direct access.	Print the actual keys; check membership.
Case or spaces	A seemingly familiar record is not found.	Inspect `repr` of the requested name.
Inconsistent vocabulary	Sections of code disagree.	Compare the exact schema names.
Lost units	Plausible number, wrong interpretation.	Trace each quantity and conversion.
Unexpected mutation	Reference data change.	Identify aliases and copy depth.
Wrong nested path	Failure at an inner access.	Inspect the selected intermediate record.
Unchecked sentinel	`None` used as a dictionary or number.	Handle the missing-data branch first.

3.17. A dictionary design checklist

Lecture-slide alignment: Week 6 source PDF, p. 88. PDF page numbers include title and divider pages.

Before writing the processing loop, state what one record represents and which fields are required. For every key, specify a stable meaning, expected type, and physical unit where relevant. Identify optional fields and decide how absence is reported. Normalize human-entered vocabulary only under a policy that preserves technical identifiers.

Next, decide whether the record is authoritative, a working copy, or a result. Ask whether an update can affect another reference through aliasing. When many comparable records are needed, use a list of records rather than inventing a collection of unrelated variable names. Finally, ask a teammate to read the schema without the implementation: can they correctly identify what each value means?

Design principle

Good dictionary design makes the data contract inspectable before the first calculation. Clear keys are useful only when the units, missing-data behavior, and mutation policy are equally explicit.

3.18. Assignment 3: preparation, not a replacement brief

Lecture-slide alignment: Week 6 source PDF, pp. 89–91. PDF page numbers include title and divider pages.

The slides suggest a course-aligned focus: trace simple recursion, compare it with iteration, and use dictionaries for named engineering data. A possible integrated structure includes numerical and non-numerical recursive functions, engineering dictionaries, a processed list of records, a clear report, and tests covering known values and boundaries.

Assignment 3 note

This is a planning framework. The presentation does not establish a final task, deadline, file format, grading breakdown, or permission to use particular assistance. Obtain those requirements from Moodle. Neither the worked exercises below nor the companion examples add mandatory submission requirements.

Planning question	Evidence to prepare
What is the objective?	Inputs, outputs, assumptions, units, and intended decision or report.
What recurs?	Function name, base cases, smaller input, and combination rule.
Why does it terminate?	A decreasing finite size measure on every valid recursive path.
What does a dictionary mean?	Keys, value types, required fields, and units.
How is absence handled?	A membership check, a suitable sentinel, or validation.
How is the result verified?	Base, near-base, normal, invalid, and independently known tests.
What is submitted?	Only the files and evidence required by the final Moodle instructions.

3.18.1. Individual understanding and responsibility

Be prepared to explain, trace, reproduce, or modify every submitted function and data operation. Before seeking assistance, write the base cases, reductions, schema, and first tests yourself. Before submission, restart the environment, run the complete program from a known starting state, inspect keys and units, and document permitted assistance as required.

The core question is not whether a program was obtained quickly. It is whether its author can explain why it stops, what it returns, what the data mean, and which evidence supports its use. A fluent description of code is weaker than a successful hand trace and a well-chosen boundary test.

3.19. Connect recursion and dictionaries

Lecture-slide alignment: Week 6 source PDF, p. 92. PDF page numbers include title and divider pages.

The source concept map separates two sides of an engineering problem. A self-similar task can lead to a base case and a smaller call. Named information can lead to a key–value dictionary. Both should lead to a tested result and an interpretable report.

The two ideas are complementary rather than inseparable. A dictionary-based lookup does not need recursion. A recursive factorial does not need a dictionary. They can be integrated when a recursive calculation processes values stored in a named record, but that integration should make the engineering task clearer, not more complicated.

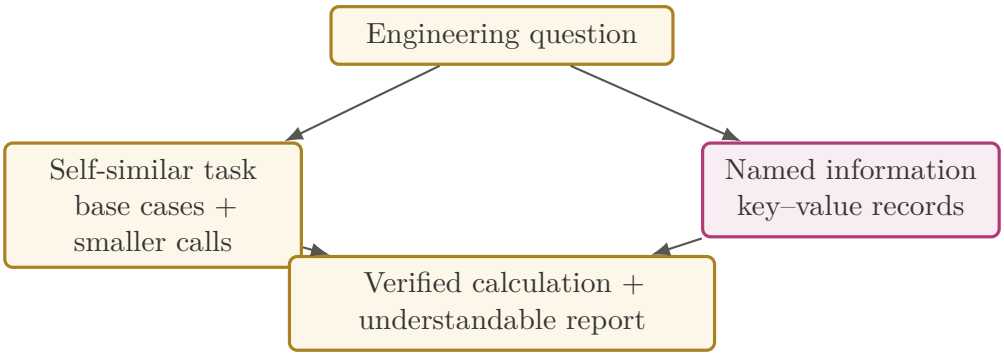


Figure 3.2.: The two complementary strands of the source Week 6 concept map, redrawn from PDF p. 92.

3.20. Lecture 18 exit ticket: a pressure-sensor record

Lecture-slide alignment: Week 6 source PDF, p. 93. PDF page numbers include title and divider pages.

The prompt requires an ID, unit, location, warning limit, calibration status, and five readings. The descriptive fields belong in a dictionary, while the repeated readings belong in a list stored under a suitable key. Unit-bearing keys such as `"warning_limit_kpa"` and `"readings_kpa"` make the intended pressure unit visible.

Worked extension of the slides

The exit ticket specifies the fields but supplies no numerical pressure dataset, location, ID, or calibration date. The following worked answer uses explicitly illustrative values. It leaves the optional calibration date absent rather than fabricating one.

Worked slide activity solution. File: `ex38_exit18.py`.

```python
# Worked solution: the exit ticket gives the required fields, not values.
# The ID, location, limit, and five readings below are illustrative.
pressure_sensor = {
    "id": "P-101",           # Text identifier; preserve leading zeros.
    "quantity": "pressure",
    "unit": "kPa",           # Unit shared by readings and warning limit.
    "location": "test line", # Illustrative location, not a course facility claim.
    "warning_limit_kpa": 200.0,
    "calibrated": True,
    "readings_kpa": [180.0, 195.0, 200.0, 205.0, 190.0]
}
# The date is optional and deliberately absent from this record.
calibration_date = pressure_sensor.get("calibration_date", None)
if calibration_date is None:
    # Do not fabricate a date from the True calibration flag.
    print("Calibration date: not recorded")
else:
    print(f"Calibration date: {calibration_date}")
# Check each measured value against the explicit illustrative limit.
for pressure_kpa in pressure_sensor["readings_kpa"]:
    if pressure_kpa > pressure_sensor["warning_limit_kpa"]:
        print(f"Warning: {pressure_kpa:.1f} kPa")
```

Code 3.16: A worked pressure-sensor schema and an optional-field lookup.

```
Calibration date: not recorded
Warning: 205.0 kPa
```

A normal test can verify that the five readings are accessible in their original order and that only 205.0 kPa exceeds the illustrative 200.0 kPa limit. A boundary test should check exactly 200.0 kPa under the chosen strict comparison. A missing-key test should omit `"calibration_date"` and confirm that the program reports `not recorded` rather than failing or inventing a date.

The flag `calibrated=True` and an absent calibration date can coexist as a data state. They answer different questions. Whether a real application permits that state is a schema and evidence requirement to be decided explicitly.

Lecture 18 in one paragraph

A dictionary associates values with keys that explain their roles. Correct use requires exact key conventions, appropriate missing-data behavior, visible units, and deliberate mutation. Nested dictionaries organize related properties, while lists of dictionaries organize repeated records. The examples retain calculated evidence separately from its presentation. Assignment 3 preparation should make the recursive reasoning, data schema, tests, and individual understanding visible, with Moodle supplying the authoritative assessment requirements.

4 Synthesis, Quick Reference, and Worked Practice

Lecture-slide alignment: Week 6 source PDF pp. 92–97, with worked practice developed from the three lectures.

4.1. The Week 6 reasoning pattern

Lecture-slide alignment: Week 6 source PDF, pp. 94, 97. PDF page numbers include title and divider pages.

The concluding message is to reduce complexity without losing meaning. Recursion reduces a problem to smaller instances and returns the information needed to complete the original task. Dictionaries associate values with names. Both rely on explicit contracts, appropriate stopping or missing-data rules, and checks that expose mistakes.

The generic recursion template in the reference slide is a design pattern, not a complete program. The helper names describe jobs that must be specified for an actual problem. The factorial, palindrome, and sum examples supply concrete versions of these jobs.

```
# TEMPLATE ONLY: the helper names below need problem-specific definitions.
def recursive_function(problem):
    # Decide whether this input can be answered without another call.
    if base_condition(problem):
        # Return the problem's correct direct or neutral result.
        return direct_answer(problem)
    # Construct a strictly simpler input that still satisfies the contract.
    smaller = reduce_problem(problem)
    # Wait for the smaller task to supply its result.
    result = recursive_function(smaller)
    # Return the complete answer for the original current input.
    return combine(problem, result)
```

Code 4.1: Conceptual template from the reference slide; helper functions are placeholders.

Task	Base case(s)	Smaller input	Combination
Factorial	$n = 0$ returns 1.	$n - 1$	Multiply by n.
Course Fibonacci	$n = 0$ or 1 returns 1.	$n - 1$, $n - 2$	Add the two returned values.
Palindrome	Length 0 or 1 returns true.	Interior substring.	Matching ends and true interior.
List sum	Empty list returns 0.	Remaining list.	Add the first value.
Membership search	Empty returns false; match returns true.	Remaining list.	Return the smaller search result.
String reversal	Length 0 or 1 returns input.	Suffix after first character.	Append the original first character.
Bisection in slides	Small residual returns midpoint.	One retained half-interval.	Return the smaller call's root estimate.

4.2. Dictionary quick reference

Lecture-slide alignment: Week 6 source PDF, p. 95. PDF page numbers include title and divider pages.

Operation	Meaning and important condition
`d = {"key":  value}`	Create a mapping with one key–value entry.
`d = {}`	Create an empty dictionary.
`d[key]`	Retrieve the value; absent key raises `KeyError`.
`d[key] = value`	Add an absent key or replace an existing value.
`key in d`	Test key membership, not value membership.
`d.get(key, default)`	Return the value or default; do not insert anything.
`d.keys()`	A view of the keys.
`d.values()`	A view of the values.
`d.items()`	A view of key–value pairs.
`for key in d:`	Iterate through keys.
`for key, value in d.items():`	Iterate through paired names and values.
`del d[key]`	Delete a known key; missing key raises `KeyError`.
`d.pop(key, default)`	Remove and return, or return a default if absent.

Technical clarification

The creation entry in the source PDF's final quick-reference table omits visible curly braces. The valid Python syntax is `d = {"key":  value}`, consistent with the complete creation examples earlier in the deck. This is an explicit presentation repair, not a change to the dictionary concept.

4.3. A Week 6 glossary

Base case
An input condition that returns a direct answer without another recursive call.

Recursive case
A nonterminal case solved using one or more smaller instances of the same problem.

Input contract
The assumptions about valid types, ranges, structure, units, and returned information.

Progress measure
A quantity that shows movement toward a reachable stopping condition, such as a nonnegative integer count or sequence length.

Call frame
The local execution context of one active function call, including its parameters and pending work.

Call stack
The collection of active frames waiting for their work to finish.

Descent
The phase in which recursive calls create smaller subproblems.

Unwinding
The phase in which completed calls return values to their waiting callers.

Recursion depth
The length of an active recursive call chain, not the total number of calls over the whole computation.

Recursion tree
A picture of individual calls and the smaller calls they make; repeated arguments may appear at multiple nodes.

Neutral value
A direct value that does not change a combination, such as zero for addition or one for multiplication.

Short-circuit evaluation
Skipping an expression when an earlier Boolean result already determines the needed logical outcome.

Residual
A calculated model output minus the desired target; its unit is the model output's unit.

Bracket
An interval with evidence that a root lies within it; in the source bisection example, continuity and opposite endpoint signs supply that evidence.

Dictionary
A mutable mapping from keys to values.

Key
An identifier used to retrieve or update an associated dictionary value.

Value
The object associated with a key; it can itself be a collection.

Schema
The agreed field names, meanings, types, units, and required or optional status of a record.

Metadata
Information that explains a dataset, such as its source, quantity, units, location, or calibration status.

Sentinel
A distinguished value such as `None` that represents a special condition, including missing information.

View
An object exposing dictionary keys, values, or pairs while reflecting the underlying mapping.

Nested dictionary
A dictionary whose values include other dictionaries.

Aliasing
Multiple names or references associated with the same mutable object.

Shallow copy
A new outer collection whose nested objects are still shared unless copied separately.

Verification
Checking the program against its intended specification and independently known cases.

4.4. Cumulative practice questions

These questions are additional reading practice based on the lecture examples, not a replacement for Assignment 3. Attempt them without running Python first. The following section provides worked answers.

1. Descent and unwinding

For the two functions in Code 1.2, predict the output of `show_a(4)` and `show_b(4)`. How many calls does each make, including the base case? Does either function return its printed sequence as a list?

2. A factorial contract and small tests

Give independent expected values for factorial inputs 0 through 5. State the behavior of the *validated slide version* for -1, 2.5, and `True`. Explain why agreement between the recursive and iterative versions is useful but not sufficient by itself.

3. A missing return

For the defective function in Code 1.9, distinguish the outcomes for inputs 0, 1, and 2. Identify the earliest failing operation for input 2 and the smallest repair.

4. Course Fibonacci

Using exactly $F(0) = F(1) = 1$, calculate $F(2)$ through $F(6)$. Draw every call for $F(3)$. Explain why drawing one shared $F(1)$ value would not accurately represent the separate function invocations.

5. Palindrome boundaries and normalization

Predict the exact-string test for `""`, `"x"`, `"abba"`, `"level"`, `"Level"`, and `"sensor"`. Then predict the phrase test for `"Never odd or even"` and `"123!"`. Which answers depend on the cleaning policy rather than only the recursive structure?

6. Reverse and test

Give expected results for recursive reversal of `""`, `"A"`, `"NU"`, and `"pump"`. Explain why the original first character must be appended *after* the reversed suffix.

7. Summation and early stopping

Trace `recursive_sum([2.0, 3.5, 4.0])`. Compare the stopping logic of a function that asks whether *any* reading exceeds a limit with a function that counts *all* exceedances. What would an empty input mean for each returned value, and what would it not prove physically?

8. Bisection evidence

In the impact-time example, what are the units of `tolerance`? Calculate the endpoint signs for 0 and 3 seconds. Does the source function stop on a small interval width? Does printing four decimal places guarantee a time error below 10^{-4} s?

9. Keys and missing data

Given `d = {"steel": 7850, "aluminum": 2700}`, explain the difference between asking whether `"steel"` is in `d` and whether 7850 is in `d`. What does `d.get("titanium")` return? Does that call add titanium to the dictionary?

10. Nested lookup debugging

A record lookup with `" Steel "` returns `None`; the next line attempts `record["density"]`. Identify the exception and the failed assumption. Explain why substituting a default density of zero is not an adequate repair.

11. Design-table verification

Use $F = 12000$ N to independently check the stresses of the source designs with areas 300, 450, and 600 mm^2. For an additional hypothetical area of 150 mm^2, what does the exact source status rule return? Does that result certify a real design? Are the original dictionaries changed by the calculation loop?

12. Integrate the two Week 6 strands

Using the source temperature-sensor record, count strict exceedances of 80.0 degrees Celsius both recursively and iteratively. Store the sensor identifier, number of readings, warning count, and method-agreement result in a separate report dictionary. Preserve the original sensor record and state a boundary test.

4.5. Worked solutions

1. Output order

`show_a(4)` prints 4, 3, 2, 1 on successive lines. `show_b(4)` prints 1, 2, 3, 4. Each makes five calls, including the call with zero. Each ultimately returns `None`; neither creates or returns a list. The difference is the location of `print(n)` relative to the recursive call, not a different sequence of descending arguments.

2. Factorial tests

The independent expectations are $1, 1, 2, 6, 24, 120$ for inputs 0 through 5. The validated slide version returns `None` for -1 and 2.5. It accepts `True` and returns 1 because of the Boolean–integer relationship discussed in the clarification. The following exercise tests against known values as well as comparing the two implementations.

Worked testing exercise. File: `ex43_comparison_tests.py`.

```python
# These small functions repeat the two algorithms for a standalone check.
def factorial_recursive(n):
    if n == 0:  # Neutral product at the base case.
        return 1
    return n * factorial_recursive(n - 1)  # Reduce, combine, return.

def factorial_iterative(n):
    result = 1  # Neutral initial product.
    for value in range(1, n + 1):
        result *= value  # Accumulate every factor through n.
    return result

# Expected values are independently known, not generated by either function.
expected = [1, 1, 2, 6, 24, 120]
for n, known_value in enumerate(expected):
    recursive_value = factorial_recursive(n)
    iterative_value = factorial_iterative(n)
    # A printed Boolean test is enough at this stage of the course.
    passed = recursive_value == known_value and iterative_value == known_value
    print(n, recursive_value, iterative_value, passed)
```

Code 4.2: Worked verification using an independent factorial table.

```
0 1 1 True
1 1 1 True
2 2 2 True
3 6 6 True
4 24 24 True
5 120 120 True
```

Both methods can agree while sharing a wrong specification or base value. Agreement therefore supplements, rather than replaces, the independently written expectations and the contract review.

3. Missing-return outcomes

The defective `f(0)` returns 1. `f(1)` calls the base case, computes 1×1, discards it, and returns `None` at the end. `f(2)` receives that `None` from its smaller call and attempts `2 * None`, causing `TypeError`. Add `return` before the multiplication in the recursive case, then retest all three inputs.

4. Fibonacci values and calls

The values are $F(2) = 2$, $F(3) = 3$, $F(4) = 5$, $F(5) = 8$, and $F(6) = 13$. Computing $F(3)$ makes five calls: the root $F(3)$; its left child $F(2)$; that child's calls $F(1)$ and $F(0)$; and the root's separate right child $F(1)$. The two $F(1)$ calls return the same value but are distinct executions. No automatic reuse occurs in the supplied function.

5. Palindrome answers

The exact-string results are true, true, true, true, false, and false, in the question's order. `"Level"` fails because `"L"` and `"l"` differ. `"sensor"` fails at its first comparison. The phrase test returns

true for `"Never odd or even"` after cleaning to `"neveroddoreven"`. It also returns true for `"123!"`, because cleaning produces the empty string. Accepting an input with no retained letters is the implemented policy, not a universal definition of meaningful input.

6. Reversal tests

The expected outputs are `""`, `"A"`, `"UN"`, and `"pmup"`. Putting the first character before the reversed suffix would not implement the desired reversal recurrence. The boundary cases verify both zero- and one-character paths.

Worked testing exercise. File: `ex45_reverse_tests.py`.

```
# Repeat the short function so this exercise is self-contained.
def reverse_recursive(text):
    if len(text) <= 1:  # Both terminal lengths must be covered.
        return text
    return reverse_recursive(text[1:]) + text[0]  # Tail first, head last.

# Each tuple pairs a test input with an independently written expectation.
tests = [("", ""), ("A", "A"), ("NU", "UN"), ("pump", "pmup")]
for text, expected in tests:
    actual = reverse_recursive(text)
    # repr makes the empty string visible in a report.
    print(repr(text), "->", repr(actual), actual == expected)
```

Code 4.3: Worked reversal tests with visible empty strings.

```
'' -> '' True
'A' -> 'A' True
'NU' -> 'UN' True
'pump' -> 'pmup' True
```

7. Sums versus existence questions

The list sum expands to $2.0 + 3.5 + 4.0 + 0 = 9.5$. A warning *existence* question can return true at the first observed violation. A warning *count* must continue to determine whether further violations occur. For an empty input, an existence scan returns false and a count returns zero. Neither answer establishes that an unobserved physical system stayed within limits.

Worked slide activity solution. File: `ex46_first_warning.py`.

```python
# Worked extension of the source's early-stopping monitoring connection.
def any_above(readings, limit):
    # Empty data have no observed exceedance; this is not proof of safety.
    if len(readings) == 0:
        return False
    # One observed exceedance is sufficient to answer this existence question.
    if readings[0] > limit:
        return True
    # Continue only when the current sample has not settled the question.
    return any_above(readings[1:], limit)

# One strictly high value exists in the source readings.
print(any_above([72.1, 73.5, 75.0, 81.2, 79.8], 80.0))
# An exact boundary value does not satisfy a strictly-greater-than test.
print(any_above([80.0], 80.0))
```

Code 4.4: Worked extension: stop when one threshold violation is sufficient evidence.

```
True
False
```

8. Bisection interpretation

The tolerance is in metres because the residual function returns height. The endpoint values are 20 m and −24.145 m. The implementation has no interval-width test: it accepts only when the absolute residual is below the supplied tolerance. Formatting to four decimal places is not an independent error bound. An analytical result within the same model provides an additional numerical check, while neither numerical agreement nor recursion validates the neglected air resistance.

9. Dictionary membership

The string `"steel"` is a key, so its membership test is true. The integer 7850 is a value rather than a key in this dictionary, so `7850 in d` is false. `d.get("titanium")` returns `None` and leaves the dictionary unchanged. A value-only membership test would need `d.values()` and would answer a different question.

10. The actual exception

Subscript access on `None` causes `TypeError`, not `KeyError`. The raw name did not satisfy the dictionary's whitespace and capitalization convention. Normalize the name under the allowed vocabulary, perform the lookup, check the sentinel, and only then retrieve a required inner property. A zero default would turn missing evidence into a plausible numerical result and could contaminate later calculations.

11. Design stress and the boundary

Because $1\,\mathrm{N/mm^2} = 1\,\mathrm{MPa}$, an independent check is $12000/300 = 40$ MPa, $12000/450 \approx 26.6667$ MPa, and $12000/600 = 20$ MPa. At 150 $\mathrm{mm^2}$, the stress is exactly 80 MPa, which passes the source's inclusive `<=` rule and receives the label `"safe"`. That is only a statement about this

simplified criterion. The calculation loop adds stress and status entries to the existing dictionaries in the list; it does not preserve those records as untouched raw inputs.

12. A worked integrated solution

The source readings contain one strict exceedance, 81.2 degrees Celsius. The function below returns zero for an empty list and contributes either one or zero for the first reading before recursively processing the remainder. The iterative version answers the same question by maintaining a counter. Both operate on the supplied data without changing it.

Worked extension of the slides

The next program is a worked integration developed from the source sensor example and concept map. It is not the official Assignment 3 solution or a new mandatory task. Its sensor values and strict warning rule come from source PDF pp. 81–82; the separate report dictionary and recursive counting function are developed here.

Worked extension of the slides. File: `ex44_monitor.py`.

```python
# Worked extension using only the source sensor's data and warning rule.
def count_above_recursive(readings, limit):
    """Count strict exceedances in a short numeric list in one unit."""
    # An empty remainder contains no further warnings.
    if len(readings) == 0:
        return 0
    # Record the current reading's contribution without modifying the list.
    if readings[0] > limit:
        first_count = 1
    else:
        first_count = 0
    # Keep the limit fixed while reducing the list length by one.
    return first_count + count_above_recursive(readings[1:], limit)

def count_above_iterative(readings, limit):
    """Count the same strict exceedances using a loop."""
    total = 0  # Start the warning counter before the loop.
    for reading in readings:
        if reading > limit:
            total += 1
    return total

# Retain the Week 6 temperature sensor and its five source readings.
sensor = {
    "id": "T-101", "unit": "C", "calibrated": True,
    "location": "heat exchanger outlet", "warning_limit": 80.0,
    "readings_c": [72.1, 73.5, 75.0, 81.2, 79.8]
}
# Read the two algorithm inputs from named fields.
readings = sensor["readings_c"]
limit = sensor["warning_limit"]
# Compare two methods on the same data without mutating it.
recursive_count = count_above_recursive(readings, limit)
iterative_count = count_above_iterative(readings, limit)
# Keep calculated evidence in a separate result dictionary.
report = {
    "sensor_id": sensor["id"],
    "number_of_readings": len(readings),
    "warning_count": recursive_count,
    "methods_agree": recursive_count == iterative_count
}
# Present every result under a meaningful label.
for key, value in report.items():
    print(f"{key}: {value}")
```

Code 4.5: Worked integration: recursive processing, an iterative check, and a named result record.

```
sensor_id: T-101
number_of_readings: 5
warning_count: 1
methods_agree: True
```

A strong boundary test uses the singleton list `[80.0]` and limit 80.0: both methods should return zero. The singleton `[81.0]` should return one. An empty list also gives zero, but the report should retain the sample count so that zero warnings is not mistaken for evidence from many

observations. The original readings remain available for comparison and further analysis.

4.6. Readiness checklist for the next week

Before moving on, explain one recursive call trace completely, including the returned values. State a base case that could be missed, a recursive path that fails to shrink, and a missing-return failure. Compare a recursive method with its loop counterpart without claiming that one form is always best.

For dictionaries, create a record with at least two types of values, retrieve a required key, handle an absent optional key, update one field, and explain whether another reference sees the change. Trace one nested access and one unit conversion. Distinguish an exact key mismatch from an absent record and from an invalid physical value.

Evidence to report

For a trustworthy Week 6 program, be able to show: the input contract; the stopping argument; one base-case or missing-key test; one independent numerical check; visible units; an explicit mutation policy; and a report that states both the result and its limitations.

A Technical Clarifications and Source Boundaries

This appendix separates technical qualifications from the attached presentation's original examples. The main text retains the source algorithms and labels. The examples below make their restrictions observable; they do not silently replace the classroom versions.

A.1. The integer check accepts Boolean flags

Lecture-slide alignment: Week 6 source PDF, p. 22. PDF page numbers include title and divider pages.

The validator in the slide uses `isinstance(n, int)`. Python treats Boolean values as a subtype of integers, so this does not exclude `True` or `False`. The original factorial code is retained. The last test below illustrates an alternative input policy that rejects Boolean flags explicitly.

Technical clarification. File: `ex39_boolean_note.py`.

```
# Technical clarification: bool is a subclass of int in Python.
print(isinstance(True, int))
print(isinstance(False, int))
# A stricter policy can reject Boolean flags separately.
n = True
# This is an alternative policy, not the original slide's validation test.
valid_count = isinstance(n, int) and not isinstance(n, bool) and n >= 0
print(valid_count)
```

Code A.1: Technical clarification of the slide's integer validation.

```
True
True
False
```

The stricter policy is appropriate when a count must not be supplied as a Boolean flag. It is not necessary to introduce type hierarchies formally in Week 6; the point is to avoid claiming that the existing test rejects every non-count object.

A.2. Unique keys and integer-key access

Lecture-slide alignment: Week 6 source PDF, p. 69. PDF page numbers include title and divider pages.

The source says that keys are unique within a dictionary. This means that equal keys refer to one current association, not that writing a repeated key in a literal necessarily raises an error. In the example below, the later value replaces the earlier value. The other examples show why an integer key is still a key and why a tuple of appropriate values can serve as an identifier.

Technical clarification. File: `ex40_key_note.py`.

```python
# Technical clarification: braces plus key: value pairs create a dictionary.
d = {"limit": 80, "limit": 90}  # The later equal key replaces the earlier value.
print(d)
# Integers may be keys; the integer is not interpreted as a list position.
channels = {4: "pressure", 0: "temperature"}
print(channels[0])  # Select key 0 even though it was inserted second.
# A tuple of hashable values is also an appropriate key.
coordinates = {(0, 1): "sensor A"}
print(coordinates[(0, 1)])
```

Code A.2: Technical clarification: key uniqueness is not a positional indexing rule.

```
{'limit': 90}
temperature
sensor A
```

A dictionary key must meet Python's hashability requirement. These examples intentionally use common simple cases. A tuple containing a list would not meet that requirement merely because its outer structure is a tuple [6].

A.3. Views and snapshots are different objects

Lecture-slide alignment: Week 6 source PDF, p. 74. PDF page numbers include title and divider pages.

The source correctly calls the method results views. A view reflects the dictionary's later changes, whereas a list made from the keys records the keys present at that moment [5].

Technical clarification. File: `ex41_views_note.py`.

```python
# Technical clarification linked to the slide's keys-view wording.
limits = {"temperature_c": 80.0}
# Store a live view and a separate list snapshot of the current keys.
keys_view = limits.keys()
keys_snapshot = list(limits.keys())
# Add another named limit after creating both objects.
limits["pressure_kpa"] = 200.0
# The view reflects the new entry; the snapshot remains unchanged.
print(list(keys_view))
print(keys_snapshot)
```

Code A.3: A keys view and a snapshot respond differently to a later insertion.

```
['temperature_c', 'pressure_kpa']
['temperature_c']
```

Do not add or remove dictionary entries while iterating over that dictionary's live views. Depending on the operation, iteration can fail or cease to describe the intended dataset. Complete the scan first or use a deliberately chosen snapshot when structural changes are required. This is a clarification of the mutable-data caution, not a new main lecture topic.

A.4. An outer copy does not duplicate a nested list

Lecture-slide alignment: Week 6 source PDF, pp. 70, 87–88. PDF page numbers include title and divider pages.

The slides recommend copies when reference data must not be overwritten. With a nested structure, the level of copying matters. A new outer dictionary still refers to the same inner objects unless they are copied too.

Technical clarification. File: `ex42_copy_note.py`.

```
# Technical clarification: a shallow copy does not copy nested lists.
raw_sensor = {"id": "T-101", "readings_c": [72.1, 73.5]}
# Copying the outer dictionary still shares the readings list.
working = raw_sensor.copy()
working["readings_c"][0] = 0.0
print(raw_sensor["readings_c"])  # The original nested list changed too.

# Reset the input, then explicitly copy the nested list we intend to edit.
raw_sensor = {"id": "T-101", "readings_c": [72.1, 73.5]}
working = raw_sensor.copy()
working["readings_c"] = raw_sensor["readings_c"].copy()
# Now modifying this numeric list does not change the raw record's list.
working["readings_c"][0] = 0.0
print(raw_sensor["readings_c"])
print(working["readings_c"])
```

Code A.4: Technical clarification of a working copy for nested sensor data.

```
[0.0, 73.5]
[72.1, 73.5]
[0.0, 73.5]
```

The first experiment changes the raw nested list unintentionally. The second explicitly copies the list that will be edited, so the raw numerical readings stay unchanged. This targeted method is sufficient for the displayed flat numeric list; it is not a claim that it duplicates arbitrarily deep nested structures.

A.5. Clarification register

PDF p.	Source feature	Treatment in these notes
11	A nonnegative strictly decreasing measure.	Qualified for finite integer-size reductions; real-valued numerical recursion needs a justified stopping threshold.
22	`isinstance(n, int)` validation.	Retained exactly; Boolean acceptance explained separately.
24	`n <= 1` product base case.	Traced to input 1, not automatically extended to an extra call at 0; negative-input behavior not called validation.
25	Recursive expression without `return`.	Distinguished `f(1)` returning `None` from `f(2)` raising `TypeError`.

PDF p.	Source feature	Treatment in these notes
27–28	Verbal residual/interval stopping; code checks only residual.	Both statements preserved and their difference identified. No unimplemented safeguards are attributed to the source function.
29	Height residual, time input.	Tolerance identified as metres; decimal display not presented as a time-error guarantee.
35–38	$F(0) = F(1) = 1$.	Preserved throughout all code, traces, tests, and solutions.
37	Shared-looking $F(1)$ node in the drawing.	Expanded into separate call occurrences in the actual execution tree; no caching implied.
40	Display-wrapped assignment expression.	Written as valid single-line simultaneous assignment without changing semantics.
49	Letter-only lowercase cleaning.	Preserved; removal of digits and empty cleaned inputs explained. No complete multilingual normalization is claimed.
69	Common key types and uniqueness.	Hashability and repeated-key replacement clarified without teaching a hashing implementation.
70	Density update from 2700 to 2710.	Described as a demonstrative update, not a verified physical correction.
77–80	Illustrative materials data.	Values retained; no alloy grade, uncertainty, temperature, or certified source invented.
83–84	`safe`/`unsafe` labels and unlabeled cost unit.	Labels interpreted only against the stated stress test; no currency or broader safety certification added.
86	Unchecked result of `get`.	Actual `NoneType` `TypeError` explained instead of calling it a direct missing-key exception.
89–91	Assignment 3 launch and planning.	Treated as a possible framework; final assessment policy remains on Moodle.
93	Pressure-sensor schema prompt without data.	Worked values explicitly labelled illustrative; optional calibration date left absent.
94	Generic recursion helpers.	Retained as a conceptual template, not presented as standalone runnable code.
95	Missing visible braces in dictionary-creation reference.	Valid braces restored explicitly, consistent with the main creation slide.

A.6. What remains outside the supplied material

The source does not provide a production-grade recursive root solver, a real materials-property database, a final Assignment 3 brief, experimental validation of the sensor rules, or a comprehensive algorithm-efficiency analysis. These notes do not present any of those as source-supported conclusions. Worked exercises and numerical checks are labelled as such; they add explanation and practice while preserving the scope.

B Source Map and Suggested Reading

B.1. How the source was used

The primary source is the attached 97-page Week 6 presentation. The main notes follow its three lectures, retain its definitions and engineering datasets, and answer its activities. Source page numbers below refer to physical PDF pages, including title and divider pages. The notes' page numbers and the slide footer numbers may differ.

The presentation names the course specification, Wang's working course manuscript, and Guttag's textbook as suggested reading on source PDF p. 96. They are listed below in that role. Supplementary Python documentation supports only the explicitly labelled implementation clarifications; it does not replace the source sequence or create new assessment requirements.

B.2. Complete slide-to-notes map

PDF p.	Source focus	Reading-note location
1	Title and course identity	Front matter, p. 2
2	Week 6 at a glance	Front matter, p. 3
3	Lecture 16 divider	Chapter 1, p. 7
4	Lecture 16 learning goals	Chapter 1, p. 7
5	Warm-up: what repeats and what becomes smaller?	Section 1.1, p. 7
6	What is recursion?	Section 1.2, p. 7
7	The two-part recursive pattern	Section 1.2, p. 7
8	A first recursive countdown	Section 1.3, p. 8
9	Trace the calls before running	Section 1.3, p. 8
10	Code before or after the recursive call	Section 1.4, p. 10
11	The base case as a contract	Section 1.5, p. 11
12	No working base case	Section 1.6, p. 11
13	Moving away from the base case	Section 1.6, p. 11
14	The factorial recurrence	Section 1.7, p. 12
15	Recursive factorial in Python	Section 1.7, p. 12
16	Factorial calls expand	Section 1.8, p. 13
17	Call stack and unfinished work	Section 1.8, p. 13
18	Unwinding the factorial result	Section 1.8, p. 13
19	Iterative factorial	Section 1.9, p. 14
20	Recursion versus iteration	Section 1.9, p. 14
21	Choosing an engineering implementation	Section 1.9, p. 14
22	Validate the recursive input contract	Section 1.10, p. 15
23	Factorial test cases	Section 1.10, p. 15
24	Product-to-one tracing activity	Section 1.11, p. 16
25	Three recursion defects	Section 1.12, p. 17
26	Recursion depth as a resource	Section 1.13, p. 17
27	Bisection as a self-similar task	Section 1.14, p. 18
28	Residual-only recursive bisection	Section 1.14, p. 18
29	Falling-object impact-time target	Section 1.15, p. 20
30	Recursion design checklist	Section 1.16, p. 20
31	Lecture 16 exit ticket	Section 1.16, p. 20
32	Lecture 17 divider	Chapter 2, p. 22
33	Lecture 17 learning goals	Chapter 2, p. 22
34	Two starting values	Section 2.1, p. 22
35	Several base cases	Section 2.1, p. 22
36	Recursive Fibonacci	Section 2.2, p. 22
37	Fibonacci call tree	Section 2.3, p. 23

PDF p.	Source focus	Reading-note location
38	Return values combine upward	Section 2.3, p. 23
39	Correctness versus efficiency	Section 2.4, p. 24
40	Iterative Fibonacci	Section 2.4, p. 24
41	Sequence-generation comparison	Section 2.4, p. 24
42	Two-base-case prediction activity	Section 2.5, p. 26
43	Recursion on non-numerical data	Section 2.6, p. 26
44	Palindrome as a recursive question	Section 2.7, p. 27
45	Recursive palindrome test	Section 2.7, p. 27
46	Trace the word level	Section 2.7, p. 27
47	Short-circuit evaluation	Section 2.8, p. 28
48	A non-palindrome rejects immediately	Section 2.8, p. 28
49	Separate text cleaning from recursion	Section 2.9, p. 28
50	Phrase normalization activity	Section 2.9, p. 28
51	First element plus remaining list	Section 2.10, p. 30
52	Recursive list sum	Section 2.10, p. 30
53	List reduction and missing-value codes	Section 2.10, p. 30
54	Recursive sequence search	Section 2.11, p. 31
55	Two search stopping cases	Section 2.11, p. 31
56	Non-shrinking string defect	Section 2.12, p. 32
57	Too-narrow palindrome base case	Section 2.12, p. 32
58	A recipe for non-numerical recursion	Section 2.13, p. 33
59	String-reversal design activity	Section 2.13, p. 33
60	One possible reversal solution	Section 2.13, p. 33
61	Four recursion questions	Section 2.13, p. 33
62	Lecture 17 exit ticket	Section 2.14, p. 34
63	Lecture 18 divider	Chapter 3, p. 35
64	Lecture 18 learning goals	Chapter 3, p. 35
65	An ambiguous positional sensor record	Section 3.1, p. 35
66	Dictionary as a mapping	Section 3.1, p. 35
67	Create the first dictionary	Section 3.2, p. 35
68	Retrieve by key	Section 3.2, p. 35
69	Keys, values, and consistent vocabulary	Section 3.3, p. 36
70	Add and update entries	Section 3.4, p. 37
71	Missing key and KeyError	Section 3.5, p. 38
72	Membership before direct access	Section 3.6, p. 38
73	Meaningful get defaults	Section 3.7, p. 39
74	Keys, values, items, and iteration	Section 3.8, p. 40
75	Formatted items table	Section 3.8, p. 40
76	Deliberate removal	Section 3.9, p. 42
77	Interactive material-density lookup	Section 3.10, p. 43
78	Nested material-property records	Section 3.11, p. 43
79	Access a nested property	Section 3.11, p. 43
80	Normalized nested lookup	Section 3.12, p. 44
81	Sensor metadata	Section 3.13, p. 45
82	Metadata plus repeated readings	Section 3.13, p. 45
83	Calculate design dictionaries	Section 3.14, p. 46
84	Print the design table	Section 3.14, p. 46
85	Choose list, tuple, or dictionary	Section 3.15, p. 48
86	Nested-lookup debugging activity	Section 3.16, p. 48
87	Common dictionary errors	Section 3.16, p. 48
88	Dictionary design checklist	Section 3.17, p. 50
89	Assignment 3 launch	Section 3.18, p. 50
90	Assignment 3 planning template	Section 3.18, p. 50
91	Individual understanding and responsibility	Section 3.18, p. 50
92	Week 6 concept map	Section 3.19, p. 51
93	Lecture 18 exit ticket	Section 3.20, p. 52
94	Recursion quick reference	Section 4.1, p. 54
95	Dictionary quick reference	Section 4.2, p. 55
96	Suggested reading and source basis	Appendix B, p. 69
97	Closing synthesis	Section 4.6, p. 64

Bibliography

[1] Y. Wang. *Programming for Engineers: Week 6 — Recursion and Dictionaries*. School of Engineering, Nazarbayev University, Fall 2026. Attached file: `ENG101_Week6_Complete_Slides.pdf`, 97 PDF pages. Primary basis of these reading notes.

[2] Nazarbayev University. *ENG 101 — Programming for Engineers: Course Specification Report*, Fall 2026, Session 6. Recommended in the source presentation.

[3] Y. Wang. *Programming for Engineers with Python*, working draft, August 2026, Sections 5.13–5.16 and 7.16–7.17. Edition and sections as recommended on source PDF p. 96.

[4] J. V. Guttag. *Introduction to Computation and Programming Using Python*, 2nd edition, Sections 4.3 and 5.6. Recommended in the source presentation.

[5] Python Software Foundation. *Python 3 documentation: Built-in Types*. Supplementary reference for Boolean/integer behavior and dictionary views. Official Python documentation. Accessed 13 September 2026.

[6] Python Software Foundation. *Python 3 documentation: Mapping Types — dict*. Supplementary reference for keys and dictionary semantics. Official mapping-type reference. Accessed 13 September 2026.

[7] Python Software Foundation. *Python 3 tutorial: More Control Flow Tools*. Supplementary reference for function calls and return behavior. Official Python tutorial. Accessed 13 September 2026.

[8] Python Software Foundation. *Python 3 documentation: sys — System-specific Parameters and Functions*. Supplementary reference for the recursion-depth safeguard. Official sys module reference. Accessed 13 September 2026.

Final preparation habit

Trace recursion on paper, explain why every valid path reaches an appropriate base case, and verify the returned values. Write dictionary keys as a documented schema. Then check the units, missing-data behavior, and evidence behind the resulting engineering report.